```
-- Scripts/main.lua
--
-- FireGame 入口：第三人称俯视角"狙击海盗"垂直切片（全自动开火，玩家只负责走位）。
-- 本文件不含玩法逻辑，只负责按依赖顺序装配各模块并驱动帧循环：
--   Config → Coast（海岸布景）→ Player（主角+俯视相机）→ Fx/Audio（反馈）
--   → Hud（数据绑定 HUD）→ Enemy（单位管理）→ Waves（波次导演）→ Turrets（友方炮塔）
--   → Shooting（自动开火）
--
-- 宿主约定：三个宿主（编辑器 GameView Play / MinigamePlayer / LuaGamePlayer）都在 C++ 侧
--   注入全局 `scene` 后再执行本脚本；场景根节点必须挂 LuaGameScript 且 Script Path 指向
--   本文件，否则 main.lua 永不执行（见 Default.scene）。脚本不创建场景，只往里补东西。
--
-- require 以 Scripts/ 为基准（EngineLuaVM 的 requirePrefixes_），故写 game.Config / core.Joystick。

local Joystick = require("core.Joystick")
local Config = require("game.Config")
local Coast = require("game.Coast")
local Player = require("game.Player")
local Fx = require("game.Fx")
local Audio = require("game.Audio")
local Hud = require("game.Hud")
local CamDebug = require("game.CamDebug")
local Enemy = require("game.Enemy")
local Waves = require("game.Waves")
local Turrets = require("game.Turrets")
local Shooting = require("game.Shooting")
local Juice = require("game.Juice")
local Grow = require("game.Grow")
local Dog = require("game.Dog")
local HpBar = require("game.HpBar")
local Voice = require("game.Voice")
local Sidebar = require("game.Sidebar")
local Tracker = require("game.Tracker")

-- ---------------------------------------------------------------------------
-- 帧率上限：真机（抖音 wasm）主循环按 Config.MAX_FPS 节流，省电/降热。
-- 必须在脚本加载期（顶层）同步设置：MinigamePlayer 在 Start() 里跑完本脚本后
-- 才去注册主循环（Application::Run 读 Engine::GetWebFrameRate），此时写入方能生效。
-- ---------------------------------------------------------------------------
do
    local engine = GetSubsystem("Engine")
    if engine and engine.SetWebFrameRate and Config.MAX_FPS and Config.MAX_FPS > 0 then
        engine:SetWebFrameRate(Config.MAX_FPS)
    end
end

-- ---------------------------------------------------------------------------
-- HUD 几何与 UI 缩放：rml 全部用 dp 单位（1dp = Config.UI_SCALE 画布像素），
-- 本文件里凡与 rml 几何同步的量（摇杆环/帽/行程/抓取半径）= rml 的 dp 基础值
-- × UI_SCALE。缩放系数按画布短边派生（Config.ComputeUIScale），启动时算一次
```

```
-- 经 rmlUi:SetScale 生效；编辑器拖 GameView 改尺寸时帧循环里重算（真机恒定）。
-----------------------------------------------------------------------
local STICK_BASE = {
    inset = 40.0,        -- GameHud.rml: #stick_ring 的 left / bottom（dp）
    size = 224.0,        -- width / height（dp；半径 112 = 行程 64 + 帽半宽 48，满偏时帽外缘恰贴环边）
    travel = 64.0,       -- 帽行程（dp）：40dp 在真机只有 22px，轻碰即满偏没量程；64dp≈36px
    knobHalf = 48.0,     -- Knob.rml 是 96x96（dp）
    grab = 260.0,        -- 起手判定半径（dp）：浮动原点式，放宽到覆盖整块左下移动区，好按
}
-- 画布像素量（= 基础值 × UI_SCALE），ApplyUIScale 每次缩放变化重算
local STICK = {}
local function ApplyUIScale(scale)
    Config.UI_SCALE = scale
    STICK.inset = STICK_BASE.inset * scale
    STICK.size = STICK_BASE.size * scale
    STICK.travel = STICK_BASE.travel * scale
    STICK.knobHalf = STICK_BASE.knobHalf * scale
    STICK.grab = STICK_BASE.grab * scale
    STICK.cx = STICK.inset + STICK.size * 0.5
end

-- 画布尺寸必须问 RmlUI 要：编辑器 GameView 里 RmlUi 的 context 是那块 backbuffer，
-- 而 Graphics:GetWidth 是整个编辑器窗口（浏览器/抖音里两者相等，兜底分支才用窗口）。
local function CanvasSize()
    local ui = GetSubsystem("RmlUI")
    if ui and ui.GetCanvasSize then
        local size = ui:GetCanvasSize()
        if size.x > 0 and size.y > 0 then
            return size.x, size.y
        end
    end
    local graphics = GetSubsystem("Graphics")
    if graphics and graphics:GetWidth() > 0 and graphics:GetHeight() > 0 then
        return graphics:GetWidth(), graphics:GetHeight()
    end
    return 0.0, 0.0
end

-- 环心（画布像素）：与 GameHud.rml 里 #stick_ring 的 left/bottom 锚位同一个公式。
-- 环按 bottom 锚边，Y 随画布高度变，调用方每帧重算。
local function StickCenter(canvasH)
    return STICK.cx, canvasH - STICK.cx
end

-----------------------------------------------------------------------
-- 中文字体：默认 Noto Sans 无 CJK 字形，中文画不出来。
-- 把 Noto Sans SC 注册为全局回退字体（fallback=true）：主字体缺字时自动落到它，
-- 英文数字仍用 Noto Sans，中英混排自然。
-- 出包前再子集化（扫描用字 → pyftsubset，子集是构建派生物，源库永远放全量），
-- 开发期直接用全量，UI 随便加字不会缺字。
```

```
-- 必须在 HUD 文档打开前完成。
-----------------------------------------------------------------------
local rmlUi = GetSubsystem("RmlUI")
if rmlUi and not rmlUi:LoadFont("Fonts/NotoSansSC-Regular.otf", true) then
    print("FireGame MARK font fallback FAILED: Fonts/NotoSansSC-Regular.otf")
end

-----------------------------------------------------------------------
-- UI 缩放与画布量：必须在 HUD/Knob 文档打开与 Player/Juice 装配前完成
-- （rml 的 dp 按 SetScale 折像素，相机垂直 FOV 按宽高比派生，两者都吃这对输入）。
-----------------------------------------------------------------------
local canvasW, canvasH = CanvasSize()
local uiScale = Config.ComputeUIScale(canvasW, canvasH)
ApplyUIScale(uiScale)
if rmlUi then
    rmlUi:SetScale(uiScale)
end
local canvasAspect = (canvasH > 0.0) and (canvasW / canvasH) or nil

-----------------------------------------------------------------------
-- 装配
-----------------------------------------------------------------------
-- 先声明，RestartGame / 回调是闭包，运行时才解引用这些 upvalue
local coast, player, fx, audio, hud, enemy, waves, shooting, juice, knob, joystick, grow, dog,
camDebug, hpBar, turrets, turretBars, voice, sidebar
-- 激励视频复活链路两状态：ad_showing=广告在途（世界 dt 压 0、吞摇杆，结果事件结算时清）；
-- reviveAdEnabled=链路总开关（宿主 SDK 可用 + Config.AD.REWARDED_UNIT_ID 非空，Build 后定值）。
-- 休眠语义：没配广告位时恒 false，结算面板根本不出现复活按钮，玩家看不到任何广告痕迹。
local ad_showing = false
local reviveAdEnabled = false

local function RestartGame()
    -- 重开埋点在 Reset 之前取数：局终数据只存在 waves 里，Reset 一跑就清零了。
    -- restart 与 game_start 分开计数，两者相除就是重开率。
    if waves then
        Tracker.Report("restart", { wave = waves.waveIndex, score = waves.score })
    end
    if enemy then
        enemy:Clear()
        enemy:SpawnStaticBarrels()
    end
    if shooting then
        shooting:Reset()
    end
    if player then
        player:Reset()   -- 满血、解除死亡/无敌态（阵亡动画是终态，不自动恢复）
    end
    if grow then
        grow:Reset()
    end
    if dog then
```

```
        dog:Reset()   -- 宠物狗回身边、清掉在空中/起身态（狗不死，不重生）
    end
    if waves then
        waves:Reset()
    end
    if turrets then
        turrets:Reset()   -- 在飞弹丸清空、冷却回首发迟、塌下来的炮管收回去（血量由 waves:Reset 复位）
    end
    if juice then
        juice:Reset()
    end
    if hud then
        hud:HideEnd()
        hud:CloseVoice()   -- 上一局开着的语音面板不带进新局（否则新局一开就定格）
        if hud.SetReviveAvailable then hud:SetReviveAvailable(false) end
    end
    if voice then
        voice:Stop()       -- 上一局的语音/试听不拖进新局
    end
    -- 广告结果事件没回来就点了重开（宿主异常等）：清 ad_showing，别让新局世界卡在暂停。
    ad_showing = false
end

-- 看广告复活的结算（激励视频确认看完后调）：这一局继续，不是换一局——
-- 战绩/成长/狗/已打掉的爆桶全保留，只回满血、清场残敌、复位主角与炮塔姿态。
-- 资格判定在 Waves:Revive 内（每局一次 + 确实处于败局），不满足则什么都不动。
local function ReviveGame()
    if not (waves and waves:Revive()) then
        return
    end
    if enemy then
        enemy:Clear()   -- 残敌清场：复活后不再接着挨上一波的围殴（静态爆桶不重摆，保持"这局继续"语义）
    end
    if player then
        player:Reset()  -- 满血、解除阵亡定格（hero 死因必须；defense 死因时无伤大雅）
    end
    if turrets then
        turrets:Reset() -- 在飞弹丸清空、塌下的炮管收回去（defense 死因必须）
    end
    if voice then
        voice:Stop()    -- 败局播报的录音/试听不拖进续战
    end
    if hud then
        hud:HideEnd()
        hud:CloseVoice()
        if hud.SetReviveAvailable then hud:SetReviveAvailable(false) end
    end
end

-- 计分/combo 集中到 Waves：玩家子弹（Shooting）与爆桶连锁（Enemy:Damage）两条路径共用；
```

```
-- 同一个汇入点也是打击感三件套的唯一触发口（击杀 = hit-stop + 震屏 + 飘字）。
-- fromChain 区分击杀来源（子弹 false / 爆炸连锁 true），只用来闸住“是否触发新爆炸”（切断自激）。
local function OnScore(result, unit, headshot, fromChain)
    if waves then
        waves:AddKill(result, unit, headshot, fromChain)
    end
    if juice and result and result.killed then
        juice:OnKill(unit, result.score)
    end
    -- 击杀喂经验→驱动局内成长（扇形弹丸数/伤害爬升）。grow 内部会经 onWeapon
    -- 把新武器数值推回 shooting、并升级时播旁白/震屏，这里只管喂。
    if grow and result and result.killed then
        grow:AddKill(unit and unit.kind)
    end
end

-- 海盗砍到某座炮塔：扣那座塔自己的血（经 Waves 统一裁决败局）+ 一记轻震提示“有人在摸防线”。
-- 但不做 hit-stop（玩家没杀东西，停时间只会困惑）。塔号与 Config.TURRETS / waves.turretHp 同下标。
local function OnTurretHit(index, dmg)
    if waves then
        waves:DamageTurret(index, dmg)
    end
    if juice then
        juice:AddTrauma(0.3)
    end
end

-- 塔被打哑：播报哪一侧失去火力（合计血条看不出“快塌的是左边那座”），不给语音、只上旁白。
local function OnTowerDown(index, x)
    if hud then
        hud:Narrate((x < 0.0) and "左炮塔失能！" or "右炮塔失能！", 2.5)
    end
end

-- 突袭型海盗砍到主角：扣主角血（经 Waves 统一裁决败局）+ 受击反馈。这一击是玩家自己
-- 遭殃，给足创伤（比防线受击重）。“砍死”则定格主角阵亡动画，否则播受击后仰。
local function OnPlayerHit(dmg)
    if waves then
        waves:DamagePlayer(dmg)
    end
    if juice then
        juice:AddTrauma(0.7)
    end
    if audio then
        audio:Play("hitFlesh")
    end
    if player then
        if waves and waves.playerHp > 0 then
            player:OnHurt()
        else
```

```
            player:Die()
        end
    end
end

local function OnGameOver(outcome, score)
    print(string.format("FireGame MARK game over: %s score=%d", tostring(outcome), score or 0))
    -- 败局且未用过复活 + 链路总开关开着：结算面板点亮「看广告复活」。
    -- 三层缺一即整条不亮（休眠链路：没配广告位/宿主不支持时按钮不存在，不是隐藏）。
    if outcome == "lose" and reviveAdEnabled and waves and waves:CanRevive()
        and hud and hud.SetReviveAvailable then
        hud:SetReviveAvailable(true)
    end
end

-- 载具交互（HUD 按钮回调）：登船/下船（player 在下方装配后才存在，这里是闭包运行时才解引用）
local function OnBoard()
    if player then player:BoardBoat() end
end

local function OnDisembark()
    if player then player:LeaveBoat() end
end

coast = Coast.Build(scene)
player = Player.Build(scene, coast.playerSpot, coast.playerBoat, canvasAspect)
-- 打击感三件套：要主角的相机（飘字投影 + 震屏偏移的宿主）与画布尺寸
juice = Juice.Build(scene, {
    camera = player.camera,
    cameraNode = player.cameraNode,
    canvasSize = CanvasSize,
})
fx = Fx.Build(scene)
audio = Audio.Build(scene)
audio:PlayBGM("Sound/back_ground/mixkit-dirty-thinkin-989.ogg", 0.25)
hud = Hud.Build(scene, {
    onRestart = RestartGame,
    -- 开始画面：收起面板、放行世界；game_start 在这里报——点击才是“用户主动决定开始”，
    -- 与 boot（加载完成）分开，冷启动流失和玩法流失才能分开算。
    onStart = function()
        if hud then
            hud:HideStart()
        end
        Tracker.Report("game_start")
        -- 侧边栏复访：冷启动查到的引导/奖励播报一直被开始画面压着，收起后才放出来
        -- （game/Sidebar.lua 的 OnGameStart）。
        if sidebar then
            sidebar:OnGameStart()
        end
        -- 补播 Waves.Build 时被跳过的“准备”旁白：倒计时从这一刻走，语音同步响，
```

```
        -- 与重开（RestartGame 里 waves:Reset 带语音）保持同一体验。
        if audio and audio.Voice then
            audio:Voice(Config.Voice.prepare)
        end
    end,
    onBoard = OnBoard,
    onDisembark = OnDisembark,
    -- 结算页分享：战绩标题带波数与分数（query 留给以后进入侧解析/看板），分享面板由宿主拉起。
    -- waves 是 upvalue，点击发生在 Build 之后很久，运行时解引用必然就绪。
    onShare = function()
        if not waves then
            return
        end
        Tracker.Report("share", { score = waves.score, wave = waves.waveIndex })
        local sdk = GetSubsystem("MinigameSDK")
        if sdk and sdk:IsAvailable() then
            sdk:Share(
                string.format("我在海盗防线守到第 %d 波，拿下 %d 分，来试试？", waves.waveIndex,
waves.score),
                "",
                string.format("score=%d&wave=%d", waves.score, waves.waveIndex))
        end
    end,
    -- 结算页看广告复活：拉起激励视频，结果经顶层 MinigameSdkResult 订阅回来在那里结算复活。
    -- 这里只做资格校验与请求：非败局/已用过/链路休眠/广告在途 → 静默不动（按钮本就不亮，
    -- 这里是防伪造/防连点的第二道闸）。世界暂停靠 ad_showing，与开始画面同一手法。
    onRevive = function()
        if ad_showing or not reviveAdEnabled then
            return
        end
        if not (waves and waves:CanRevive()) then
            return
        end
        local sdk = GetSubsystem("MinigameSDK")
        if not (sdk and sdk.IsAvailable and sdk:IsAvailable()) then
            return
        end
        ad_showing = true
        if hud and hud.SetReviveAvailable then
            hud:SetReviveAvailable(false)   -- 广告期间按钮收起，防连点重入
        end
        Tracker.Report("ad_request", { wave = waves.waveIndex, score = waves.score })
        sdk:ShowRewardedAd(Config.AD.REWARDED_UNIT_ID)
    end,
    -- 自定义语音：面板只管 UI，录/试听/换音色转发给 Voice（本函数之后才 build，闭包运行时解引用）
    onVoiceRecord = function(slot)
        if voice then voice:Record(slot) end
    end,
    onVoicePreview = function(slot)
        if voice then voice:Preview(slot) end
    end,
```

```
    onVoiceClear = function(slot)
        if voice then voice:Clear(slot) end
    end,
    onVoiceFilter = function()
        if voice and hud then
            hud:SetVoiceFilterName(voice:CycleFilter())
        end
    end,
    -- 侧边栏引导弹窗：「去侧边栏」拉起宿主面板 + 持久化已引导；「知道了」只持久化。
    -- 持久化与面板拉起都在 game/Sidebar.lua，这里只转发点击。
    onSidebarGo = function()
        if sidebar then sidebar:GoSidebar() end
    end,
    onSidebarClose = function()
        if sidebar then sidebar:DismissGuide() end
    end,
})
-- 结算页分享入口：与语音入口同源的宿主能力判定（编辑器/桌面没有分享通路）。
do
    local sdk = GetSubsystem("MinigameSDK")
    if sdk and sdk:IsAvailable() and hud.SetShareAvailable then
        hud:SetShareAvailable(true)
    end
    -- 复活链路总开关：宿主可用 + 广告位已配。空 ID = 休眠（按钮永不出现）。
    -- 将来有 HTTPS 备案域名后，远程拉到的 adUnitId 在这里覆盖本地常量即可升级免重审。
    reviveAdEnabled = (sdk and sdk:IsAvailable()
        and (Config.AD.REWARDED_UNIT_ID or "") ~= "") and true or false
end
-- 自定义语音（麦克风录一句→变声回放到失守/连杀）：必须在 Hud 之后建（面板状态推给它）、
-- Waves 之前建（失守与连杀触发点要拿它回放）。宿主不支持时它自身全部静默，UI 也不显示入口。
voice = Voice.Build(scene, { hud = hud })
-- 侧边栏复访（抖音必接合规）：必须在 Hud 之后建（引导弹窗/旁白推给它）、waves 建好前
-- 只需 getWaves 回调（结果最早下一帧回来时 waves 已在）。编辑器无 SDK 时整个静默。
sidebar = Sidebar.Build(scene, {
    hud = hud,
    getWaves = function() return waves end,
})
-- 相机参数调试面板（仅 Config.DEBUG_CAM_PANEL=true）：右上角一颗按钮，真机上实时拖相机常量
-- 并把可粘回 Config.lua 的数值打到日志。必须在 player 之后建（它要调 player:ApplyCameraFit）。
-- 开关为 false 时下面整块不存在——上线包里连 UI 组件都不会创建。
if Config.DEBUG_CAM_PANEL then
    camDebug = CamDebug.Build(scene, player)
end
enemy = Enemy.Build({
    scene = scene, fx = fx, audio = audio, player = player,
    onScore = OnScore, onTurretHit = OnTurretHit, onPlayerHit = OnPlayerHit,
})
enemy:SpawnStaticBarrels()
-- 宠物狗：友方伴随单位，独立于 enemy.units（不被玩家子弹/波次/爆桶波及）。需 player+enemy，
-- 故在两者之后建；建完回填 enemy.dog，使大船炮弹落地时能把狗炸飞（Enemy:CannonImpact）。
```

```
dog = Dog.Build({ scene = scene, player = player, enemies = enemy, audio = audio })
enemy.dog = dog
waves = Waves.Build({ enemy = enemy, audio = audio, hud = hud, voice = voice, onGameOver =
OnGameOver,
    onWaveCleared = function() if dog then dog:GoRest() end end })
-- 友方炮塔：需 enemy（索敌）+ waves（血量）+ coast.turrets（句柄），故在两者之后建：
-- 建完回填 enemy.turrets，使冲塔型海盗能问“最近哪座塔还活着、该走到哪”（与 dog 同一条路子）。
turrets = Turrets.Build({
    scene = scene, handles = coast.turrets, enemies = enemy, waves = waves,
    fx = fx, audio = audio, onTowerDown = OnTowerDown,
})
enemy.turrets = turrets
-- 主角头顶 3 格血条：一份 RmlUI 文档渲染进离屏纹理、贴在跟随主角的 3D 面板节点上
-- （引擎原生的 UI 挂 3D 通路，不是世界坐标转屏幕坐标）。必须在 waves 之后建：
-- 血量的唯一真相在 waves.playerHp（Player.hp 只是个旧快照，别接上去）。
hpBar = HpBar.Build(scene, {
    name = "PlayerHpBar",
    cameraNode = player.cameraNode,
    getAnchorPos = function() return player.node:GetWorldPosition() end,
    -- 血量刻度读 waves.playerMaxHp（侧边栏回归增益抬上限时血条同刻度放大），
    -- 异常缺字段时回落 Config 值。
    getHp = function() return waves.playerHp, waves.playerMaxHp or Config.PLAYER_HP end,
    overhead = Config.HP_BAR.overhead,
    isHidden = function() return player.dying end,
})
-- 两座炮塔头顶血条：复用主角那套世界面板（HpBar.lua）但走 ratio 模式（连续进度条 UI/TurretHpBar.rml，
-- 绿色）——砍塔是持续掉血，3 格太粗。锚点换成每座塔的节点、血量真相换成 waves.turretHp。
-- 塔不动，但面板仍每帧跟相机对齐朝向（相机以后若可转这里不用改）。
-- 重开不重建 Coast（RestartGame 只 Reset），所以这里捕的塔节点引用整局有效。
turretBars = {}
for i, tw in ipairs(turrets.towers) do
    turretBars[#turretBars + 1] = HpBar.Build(scene, {
        name = "TurretHpBar_" .. tostring(i),
        mode = "ratio",
        cameraNode = player.cameraNode,
        getAnchorPos = function() return tw.node:GetWorldPosition() end,
        getHp = function() return waves:GetTurretHp(i) end,
        overhead = Config.TURRET_HP_BAR.overhead,
        worldW = Config.TURRET_HP_BAR.worldW,
        worldH = Config.TURRET_HP_BAR.worldH,
    })
end
shooting = Shooting.Build({
    scene = scene, player = player, fx = fx, audio = audio, hud = hud,
    enemies = enemy, onScore = OnScore,
})
-- 局内成长：Build 会调 Reset 推一次开局武器数值（1 发 1 伤）；之后每次升级回调
grow = Grow.Build({
    hud = hud,
    onWeapon = function(snapshot, isInitial)
        shooting:SetWeaponStats(snapshot.pellets, snapshot.damage, snapshot.pierce)
```

```
        -- 弹丸越多爆炸会越频繁（combo 靠击杀数涨），达阈值后把间隔抬到 BIG 档补偿，防后期滚雪球。
        if waves then waves:SetBlastEvery(Config.ComboBlastEvery(snapshot.pellets)) end
        if hud then
            hud:SetWeaponText("弹丸 x" .. snapshot.pellets)
            -- 开局不提醒；真正升级时把"变强"做成一次可听可见的反馈（飘字旁白 + 轻震）
            if not isInitial and grow and grow.level > 1 then
                hud:Narrate("升级！弹丸 x" .. snapshot.pellets, 2.0)
                if juice then juice:AddTrauma(0.4) end
            end
        end
    end,
})

-- 摇杆帽：单独一份文档，元素级样式 Lua 改不了，只能用文档级 SetPosition。
-- 注意：SetPosition 实质是写文档根元素的 left/top，所以 Knob.rml 的 body 必须带
-- position: absolute；static 下 left/top 会被布局直接忽略（每帧 SetPosition 静默失效，帽不跟手）。
-- 左摇杆驱动主角走位（见下方 player:Move）；该区域的触摸不参与射击锁定，所以拖它只动人不转视角。
knob = scene:CreateComponent("RmlUIComponent")
knob:SetResource("UI/Knob.rml")

local stickCx, stickCy = StickCenter(canvasH)
joystick = Joystick:New({ cx = stickCx, cy = stickCy, radius = STICK.travel, grab = STICK.grab })

local input = GetSubsystem("Input")
local time = GetSubsystem("Time")

print(string.format("FireGame MARK boot: canvas=%.0fx%.0f uiScale=%.2f aspect=%.2f
tower=(%.1f,%.1f,%.1f)",
    canvasW, canvasH, uiScale, canvasAspect or 0.0,
    Config.TOWER_POS.x, Config.TOWER_POS.y, Config.TOWER_POS.z))

-- 引擎侧启动事件：画布尺寸是机型分布的主要维度（低端机占比看这里）。
-- 冷启动毫秒在 JS 侧（loading.js 的 boot），这里不重复。
Tracker.Report("game_boot", {
    w = math.floor(canvasW or 0),
    h = math.floor(canvasH or 0),
    fpsCap = Config.MAX_FPS or 0,
})

-- 激励视频广告结果（BEGINFRAME 边界派发，与 Voice 的 MinigameSdkResult 同一事件）：
-- 只认 REWARDED_AD 且确有广告在途的回包。看完(Success 且仍可复活)→复活续战；
-- 中途关掉/宿主报错→回结算面板（每局一次的资格还在，可以再点；抖音侧广告自身有频控）。
-- 无论成败必须清 ad_showing，否则世界永远卡在暂停。
SubscribeToEvent("MinigameSdkResult", function(data)
    if data.Kind ~= MINIGAME_REQUEST_KIND.REWARDED_AD or not ad_showing then
        return
    end
    ad_showing = false
    if data.Success and waves and waves:CanRevive() then
        Tracker.Report("ad_complete", { wave = waves.waveIndex, score = waves.score })
        ReviveGame()
```

```
    else
        if data.Success then
            -- 看完了但败局已被重开/复活抢先——极少见的时序交叠，只记日志不发奖。
            print("Ad: rewarded ad done but run is no longer revivable")
        else
            -- 宿主回来的失败原文（中途关闭/加载失败/无填充）进日志：真机上只有这一条线索。
            print("Ad: rewarded ad failed: " .. tostring(data.Payload or ""))
        end
        Tracker.Report("ad_skip", { wave = (waves and waves.waveIndex) or 0 })
        if hud and hud.SetReviveAvailable and waves and waves:CanRevive() then
            hud:SetReviveAvailable(true)
        end
    end
end)

-----------------------------------------------------------------------
-- PC / 编辑器验证用键盘走位：WASD + 方向键，与左摇杆同一出口。
-- 注意：编辑器 GameView 必须先在其内部点一下抢占输入，游戏才收得到键鼠。
-- 触屏/抖音真机无键盘，KeyboardVector 恒返回 0，不影响摇杆。
-----------------------------------------------------------------------
local function KeyboardVector()
    if not (KEY and input) then return 0.0, 0.0 end
    local x, y = 0.0, 0.0
    if input:GetKeyDown(KEY.D) or input:GetKeyDown(KEY.RIGHT) then x = x + 1 end
    if input:GetKeyDown(KEY.A) or input:GetKeyDown(KEY.LEFT) then x = x - 1 end
    if input:GetKeyDown(KEY.S) or input:GetKeyDown(KEY.DOWN) then y = y + 1 end
    if input:GetKeyDown(KEY.W) or input:GetKeyDown(KEY.UP) then y = y - 1 end
    return x, y
end

-----------------------------------------------------------------------
-- 帧循环
-----------------------------------------------------------------------
local seaScrollT = 0.0

-- 性能诊断：由 Config.PERF_OVERLAY 总开关（默认 false = 上线态，不显不刷日志）。
-- 真机（抖音）无引擎 DebugHud（SystemUI 已禁用），靠屏幕上的 perfText Overlay + 每 3 秒一条 print 看指
标。
local perfDiagOn = Config.PERF_OVERLAY
local perfAccum = 0.0
local perfRenderer = nil
local function PerfCountUnits()
    local pirates, boats = 0, 0
    if enemy and enemy.units then
        for _, u in ipairs(enemy.units) do
            if u.alive then
                if u.kind == "pirate" then
                    pirates = pirates + 1
                elseif u.kind == "boat" or u.kind == "bigboat" then
                    boats = boats + 1
                end
```

```
            end
        end
    end
    return pirates, boats
end

SubscribeToEvent("Update", function(data)
    local dt = data.TimeStep
    -- 打击感帧首：推进 hit-stop 真实倒计时，拿到本帧世界 dt（定格期间被缩到 5%）。
    -- 引擎侧骨骼动画/粒子不受它影响（走自己的 Frame::timeStep），定格里死亡动画
    -- 照常播——这正是要的效果。世界 dt 只眼世界逻辑：海面滚动/开火/敌人/波次/主角。
    local worldDt = juice:BeginFrame(dt)
    -- 开始画面/语音面板/侧边栏引导都是模态的：开着的期间把世界 dt 压成 0（与 hit-stop 同一手法，粒子/骨骼动画
    -- 走引擎自己的 timeStep 不受影响）。开始画面盖着时波间倒计时不走——开局由玩家点击放行；
    -- 录音上限 3 秒，不暂停就是“调个音色被偷家”。激励视频广告在途（ad_showing）同理：
    -- 全屏视频盖住游戏，回来前不能刷怪/掉血/走位。
    if (hud and (hud:IsStartModal() or hud:IsVoiceModal() or hud:IsSidebarGuideModal())) or ad_showing then
        worldDt = 0.0
    end

    -- 海面波纹流动：滚动法线贴图 UV（xy 的平铺密度与 Water.xml 的 UOffset/VOffset 同源，
    -- 提成了 Config.SEA_TILE，改密度只动那一处；w 是平移量）。
    -- M_Default 没有内置滚动，这里是给“静态平铺”加上时间维度的全部代码。
    seaScrollT = seaScrollT + worldDt
    coast.seaMaterial:SetShaderParameter("UOffset", Vector4(Config.SEA_TILE, 0.0, 0.0, seaScrollT * Config.SEA_SCROLL_U))
    coast.seaMaterial:SetShaderParameter("VOffset", Vector4(0.0, Config.SEA_TILE, 0.0, seaScrollT * Config.SEA_SCROLL_V))

    -- 环底锚，画布高度可能随 GameView 拖拽变化，每帧重算环心
    local cw, ch = CanvasSize()
    if cw > 0 then
        if cw ~= canvasW or ch ~= canvasH then
            -- 尺寸变了（编辑器拖 GameView；真机恒定不进这分支）：重算 UI 缩放并同步。
            -- 引擎只在 ApplyScreenMode 时按 dpiScale 重设 SetScale（Engine.cpp），
            -- GameView 拖拽不走那条路，这里的重设不会被盖掉。
            canvasW, canvasH = cw, ch
            local scale = Config.ComputeUIScale(cw, ch)
            ApplyUIScale(scale)
            if rmlUi then
                rmlUi:SetScale(scale)
            end
            joystick:SetRadius(STICK.travel, STICK.grab)
        end
        joystick:SetCenter(StickCenter(ch))
    end

    -- 摇杆（视觉 + 抓取区判定）
    joystick:Update(input)
    local stickX, stickY = joystick:GetVector()
    -- 键盘走位优先覆盖摇杆（PC/编辑器验证用）
    local kx, ky = KeyboardVector()
```

```
    if kx ~= 0.0 or ky ~= 0.0 then
        stickX, stickY = kx, ky
    end
    -- 开始画面/相机调试面板/语音面板/侧边栏引导展开时吞掉走位（摇杆与键盘一起吞，放在两者合成之后）：
    -- 开始画面盖着时人物不该提前乱跑；小画布（编辑器 GameView）下面板会盖住左下抓取区，否则拖滑条的
    -- 同时人物在走，构图边变边看，调不准；语音面板同理，录音时摇杆帽乱跑看着像卡了；
    -- 激励视频在途时全屏广告盖着画面，触摸归宿主，摇杆帽也不该跟着残留输入乱动。
    if (hud and hud:IsStartModal()) or (camDebug and camDebug:IsModal()) or (hud and
(hud:IsVoiceModal() or hud:IsSidebarGuideModal()))
        or ad_showing then
        stickX, stickY = 0.0, 0.0
    end

    -- 全自动开火 + 装弹计时（世界 dt：定格期间不吐弹，弹与定格是同一套时间）
    shooting:Update(worldDt)

    -- 玩法与世界
    enemy:Update(worldDt)
    turrets:Update(worldDt)
    dog:Update(worldDt)
    waves:Update(worldDt)
    -- 粒子的模拟在引擎侧走真实 timeStep（Lua 改不了），所以寿命回收也用真实 dt 对齐，
    -- 否则定格期间粒子播完了节点还被扣着不收。
    fx:Update(dt)
    -- 左摇杆走位（屏幕绝对方向），重算俯视相机位置。
    -- 阵亡后锁住走位（player:Move 会把人拉回直立，与阵亡定格冲突）。
    -- 相机看向 -z，屏幕右对应世界 -x，故水平输入取反；竖直不反（屏幕上=朝海 -z，与 stickY<0 一致）。
    if not player.dying then
        player:Move(-stickX, stickY, worldDt)
    end
    player:Update(worldDt)

    -- 打击感帧尾：飘字投影上屏 + 震屏叠加到相机（必须在 player:Update 摆好相机
    -- 基础姿态之后，否则偏移被覆盖）。
    juice:EndFrame(dt)

    -- 头顶血条：排在震屏之后，才能拿到本帧最终的相机姿态做朝向对齐
    -- （放前面会用抖之前的姿态，血条比画面慢一帧）。
    hpBar:Update(dt)
    -- 炮塔头顶血条：同样排在震屏之后取本帧最终相机姿态。塔哑（血量归零）时整块自动摘掉，
    -- 重开 waves:Reset 把血补满后下一帧又亮回来。
    for _, bar in ipairs(turretBars) do
        bar:Update(dt)
    end

    -- 载具交互：两个按钮独立判定、互斥显示（条件本身互斥，显式双保险防止
    -- 遗留显示——旧版 if/elseif 只在 else 里隐藏，下船瞬间走不到 else，
    -- disembark 会和弹出的 board 同屏）。步行且靠近船→登船；在船上→下船（
    -- 不限制位置，随时可下）。
    if (not player.inBoat) and player:IsNearBoat() then
        hud:ShowButton("board")
```

```
    else
        hud:HideButton("board")
    end
    if player.inBoat then
        hud:ShowButton("disembark")
    else
        hud:HideButton("disembark")
    end

    hud:Update(dt)

    -- 摇杆帽回写：锚在固定外环中心，只按偏转量在行程内偏移。GetVector 幅值已钳 ≤1，故帽始终
    -- 贴在外环附近、不再跟着手指拖到屏幕任意处（用未取反的 stickX 保证与手指同向）。
    -- SetPosition 写的是帽文档左上角的画布像素，减半宽让圆心落在"环心 + 偏转量*行程"上。
    local ccx, ccy = StickCenter(canvasH)
    knob:SetPosition(Vector2(ccx + stickX * STICK.travel - STICK.knobHalf,
        ccy + stickY * STICK.travel - STICK.knobHalf))

    -- 性能诊断（Config.PERF_OVERLAY 开启时）：每 3 秒一次。drawcalls/tris 是引擎设备级真值
    -- （上一帧提交量），pirates/boats 是当前存活同屏数——同时喂给屏幕 Overlay 与日志。
    if perfDiagOn then
        perfAccum = perfAccum + dt
        if perfAccum >= 1.0 then
            perfAccum = 0.0
            if not perfRenderer then
                perfRenderer = GetSubsystem("Renderer")
            end
            local pirates, boats = PerfCountUnits()
            local draws, tris, fps = -1, -1, -1
            if perfRenderer then
                if perfRenderer.GetNumDrawCalls then draws = perfRenderer:GetNumDrawCalls() end
                if perfRenderer.GetNumPrimitives then tris = perfRenderer:GetNumPrimitives() end
            end
            if time and time.GetFramesPerSecond then fps = time:GetFramesPerSecond() end
            -- Overlay 一行：FPS / drawcall / 三角面(百万) / 同屏海盗·船
            if hud and hud.ShowPerf then
                -- 相机调试面板展开时让位：两者都想占屏幕上部，同屏只会互相盖掉；
                -- 帧数在那块面板自己的状态行里看（见 game/CamDebug.lua）。
                hud:ShowPerf(not (camDebug and camDebug:IsModal()))
                hud:SetPerf(string.format("FPS: %.0f | DC: %d | TRI: %.1fM | P:%d B:%d",
                    fps, draws, tris / 1048576.0, pirates, boats))
            end
            -- print(string.format("FireGame PERF: pirates=%d boats=%d drawcalls=%d tris=%d
fps=%.1f",
            --     pirates, boats, draws, tris, fps))
        end
    end
end)

--------------------------------------------------------------------------
-- 上半身扭转（Player:ApplyAim）：必须踩在"动画已应用、蒙皮未计算"的窗口上，
```

```
-- 即 Octree 更新完 drawable 的 SceneDrawableUpdateFinished——引擎注释钦定的
-- 自定义动画/IK 时点（事件带 TimeStep）。放进上面的普通 Update 里会被动画每帧
-- 覆写骨骼，扭转永远看不见。
--------------------------------------------------------------------------
SubscribeToEvent("SceneDrawableUpdateFinished", function(_data)
    if player then
        player:ApplyAim()
    end
end)
--
-- core/Joystick.lua —— 虚拟摇杆输入节点
--
-- 职责：把一次触摸（桌面/编辑器 Play 下退化为鼠标左键拖拽）换算成归一化方向向量。
-- 它不画任何东西——摇杆的视觉在 Data/UI/GameHud.rml（底环）与 Data/UI/Knob.rml（帽）
-- 里，由 UI 编辑器维护；本模块只负责输入判定，帽的位置由 main.lua 每帧回写。
--
-- 坐标系约定
--   * 所有几何量用"画布像素"表达（= rml 里的 dp 基础值 × UI_SCALE；rml 全面采用
--     dp 单位后，RmlUi SetScale 只负责把 dp 折成像素，本模块的量纲始终是折好的
--     像素，缩放系数由 main.lua 统一注入，见其 STICK_BASE/ApplyUIScale）。
--   * 触摸/鼠标坐标本来就是画布像素（原点在左上，y 向下），拿来即用。
--   * 输出的 y_ 沿用屏幕语义（向上为负），由调用方决定怎么映射到世界轴。
--
-- 与 GameHud.rml 的同步点：环心 = #stick_ring 的 left + width/2 与
--   "画布高度 - bottom - height/2"。高度那项随画布尺寸变化（body 是
--   .fullscreen，环按 bottom 锚边），所以由持有者调 SetCenter 每帧写入，
--   缺省值只是 768x1024 档下的读数。
--

---@class Joystick 浮动摇杆（按下即在该处生成原点，拖拽量=当前指针-原点）
local Joystick = {}
Joystick.__index = Joystick

-- 鼠标充当摇杆指针时的哨兵 ID（真实 touchID 从 0 起）。
local MOUSE_ID = -1

--- 构造。config 字段全部可选，缺省值即 GameHud.rml 当前的布局。
---@param config? { cx?: number, cy?: number, radius?: number, grab?: number, dead?: number }
---@return self
function Joystick:New(config)
    config = config or {}
    local self = setmetatable({}, Joystick)
    self.cx_ = config.cx or 152.0      -- 环心 X（画布像素）
    self.cy_ = config.cy or 872.0      -- 环心 Y（画布像素，原点左上，随 SetCenter 变）
    self.radius_ = config.radius or 64.0  -- 帽可移动的最大半径（画布像素，留在环内）
    self.grab_ = config.grab or 260.0  -- 起手判定半径（画布像素）：浮动原点式，放宽到覆盖整块左下移动区，好按
    self.dead_ = config.dead or 0.12   -- 死区，避免静止时帽微抖
    self.x_ = 0.0
    self.y_ = 0.0
    self.active_ = false
```

```
    self.ptrId_ = nil                   -- 当前锁定指针（手指 ID，或鼠标 MOUSE_ID 哨兵）
    self.acx_ = self.cx_                -- 浮动摇杆原点（起手瞬间落在哪就以哪为中心）
    self.acy_ = self.cy_
    return self
end

--- 环心随画布宽高比重定位（底锚元素必须跟着走，否则判定的圆和画出来的圆错位）。
---@param cx number
---@param cy number
function Joystick:SetCenter(cx, cy)
    self.cx_ = cx
    self.cy_ = cy
end

--- 行程/抓取半径重设（编辑器拖 GameView 改 UI_SCALE 后由 main 同步；真机缩放恒定不调）。
---@param radius number 帽行程（画布像素）
---@param grab number|nil 起手判定半径（画布像素）
function Joystick:SetRadius(radius, grab)
    self.radius_ = radius
    if grab then
        self.grab_ = grab
    end
end

--- 判断采样点（画布像素）是否落在抓取范围内。
---@param dx number 画布像素 X
---@param dy number 画布像素 Y
---@return boolean
function Joystick:InGrabArea(dx, dy)
    local ox = dx - self.cx_
    local oy = dy - self.cy_
    return math.sqrt(ox * ox + oy * oy) <= self.grab_
end

--- 写入一次采样点（画布像素），得到归一化方向。ocx/ocy 是本次拖拽的浮动原点。
---@param ocx number
---@param ocy number
---@param dx number
---@param dy number
function Joystick:Apply(ocx, ocy, dx, dy)
    local ox = (dx - ocx) / self.radius_
    local oy = (dy - ocy) / self.radius_
    local len = math.sqrt(ox * ox + oy * oy)
    if len > 1.0 then
        -- 原点追手：拖出最大行程时原点向指针方向推进（vector 仍 clamp 满偏），
        -- 让手指继续拖时偏转相对推进后的原点重新计量、方向与量程保持灵敏（纯输入手感；帽另锚固定环心、不贴
指针）。
        local nx, ny = ox / len, oy / len
        local push = (len - 1.0) * self.radius_
        self.acx_ = ocx + nx * push
        self.acy_ = ocy + ny * push
```

```
        ox, oy = nx, ny
        len = 1.0
    end
    if len < self.dead_ then
        ox = 0.0
        oy = 0.0
        len = 0.0
    end
    self.x_ = ox
    self.y_ = oy
    self.length_ = len
    self.active_ = len > 0.0
end

--- 每帧采样。input 是引擎 Input 子系统，坐标全是画布像素，无需换算。
---@param input Input
function Joystick:Update(input)
    -- 1) 手指优先：找回已锁定的那根（整屏任意位置都继续跟）；否则看新手指是否落在移动区。
    local numTouches = input:GetNumTouches()
    local tracked = nil
    local fresh = nil
    for i = 0, numTouches - 1 do
        local touch = input:GetTouch(i)
        if touch then
            local dx = touch.position.x
            local dy = touch.position.y
            if touch.touchID == self.ptrId_ then
                tracked = { dx = dx, dy = dy, id = touch.touchID }
            elseif not fresh and self:InGrabArea(dx, dy) then
                fresh = { dx = dx, dy = dy, id = touch.touchID }
            end
        end
    end

    local sample = tracked or fresh

    -- 2) 无手指时桌面/编辑器用鼠标左键顶上。鼠标坐标是轮询系统全局态，聚焦 GameView 即可用。
    --    已锁定为鼠标时不按区域门（按住拖到哪都跟）；仅未激活时要求按下落在移动区起手。
    if not sample then
        local MOUSEB_LEFT = MOUSEB and MOUSEB.LEFT or 0
        if input:GetMouseButtonDown(MOUSEB_LEFT) then
            local mouse = input:GetMousePosition()
            local dx = mouse.x
            local dy = mouse.y
            if self.ptrId_ == MOUSE_ID then
                sample = { dx = dx, dy = dy, id = MOUSE_ID }
            elseif self.ptrId_ == nil and self:InGrabArea(dx, dy) then
                sample = { dx = dx, dy = dy, id = MOUSE_ID }
            end
        end
```

```
    end

    if sample then
        -- 只有新指针（与当前锁定不同）才重设浮动原点；同一指针按住续拖则沿用原点，
        -- 避免一起手就因离固定环心太远而直接打满（之前“勉强能动/很难拖”的根因）。
        if sample.id ~= self.ptrId_ then
            self.acx_ = sample.dx
            self.acy_ = sample.dy
            self.ptrId_ = sample.id
        end
        self:Apply(self.acx_, self.acy_, sample.dx, sample.dy)
        return
    end

    self.ptrId_ = nil
    self.active_ = false
    self:Release()
end

--- 归位：方向清零，等下一次按下。
function Joystick:Release()
    self.x_ = 0.0
    self.y_ = 0.0
    self.length_ = 0.0
    self.active_ = false
end

---@return number, number 归一化方向（x 向右为正，y 向下为正）
function Joystick:GetVector()
    return self.x_, self.y_
end

--- 当前是否有指针按住摇杆（不要求已产生偏转）。
---@return boolean
function Joystick:IsEngaged()
    return self.ptrId_ ~= nil
end

---@return boolean 当前是否有有效输入
function Joystick:IsActive()
    return self.active_
end

return Joystick
-- Scripts/game/Config.lua
--
-- 全游戏的数值、关卡几何与资源路径单点。调难度、调手感、挪布景都只改这一个文件，
-- 其它模块一律从这里读，不写魔数。
--
-- 坐标约定：+z 是内陆（主角与防线在这侧），-z 是外海（敌人从这侧来），y 向上。
```

```
-- 水线不是一个 z，而是 Config.ShoreZ(x) 一条曲线（见下方 SHORE 段）：它是涉水/靠岸/走位
-- 判据与海面铺位的真源，防线在 DEFENSE_Z，塔在 TOWER_POS。（沙滩网格已删，改由场景手摆 prefab。）
-- 单位是米（rbfx 约定 1 单位 = 1 米，FBX 导入已按 unit_meters 自动换算）。

---@class GameConfig 只读配置表，不实例化
local Config = {}

-----------------------------------------------------------------------
-- 关卡几何
-----------------------------------------------------------------------
Config.SEA_Y = -0.20                -- 海平面高度
Config.SEA_SIZE = 400.0             -- 海面 Plane 边长，够大到雾里消失
Config.SEA_TILE = 24.0              -- 法线噪声平铺密度（400m 海面分 24 格，约 17m/格，舒缓涌浪；
                                    -- 越大波纹越细碎，改动会同步 Water.xml 与 main.lua 滚动公式）
Config.SEA_SCROLL_U = 0.03          -- 海面波纹 UV 滚动速度（UV/秒；米速 = 值×SEA_SIZE，现 12m/s）
Config.SEA_SCROLL_V = 0.02          -- 两个轴取不同速避免波纹整体平移的“传送带”感
-----------------------------------------------------------------------
-- 海岸线：一个 z 标量 → Config.ShoreZ(x) 一条曲线（海岸线的唯一真源）
--
-- 之前水线是常量 SHORE_Z=-0.5，海与沙是两块轴对齐的大 Plane 在 z=-0.5 对接，
-- 所以水陆交界在屏幕上是一条字面的直线——一眼就是“程序生成的占位”。岸线本来就不直，
-- 把它做成曲线不是为了好看而好看，是因为它同时是 5 个玩法判据的输入（涉水 y、狗活动
-- 下界、船靠岸线、玩家走位下界、炮塔警戒线）：几何与玩法同源才不会出现“看着已上岸、
-- 判据还在水里”的那类脱节。
--
-- 唯一真源纪律：控制点只有 SHORE_PTS 这一张表，上面那 5 个消费点与 Coast 的沙滩网格
-- 全从 Config.ShoreZ(x) 读，任何地方都不许再抄一个“-0.5”。
-- 控制点按 x 递增；区间外按端值平推（钳制而不是外推斜率，否则沙滩边界会在远端翘飞）。
-- 岬（凸向海）与湾（凹向陆）交替，峰谷差约 4m——幅度调大到这个量级才能在俯视镜头下
-- 真正看出“弯曲”，之前 ±0.7m 的单一缓拱只会被当成一条斜直线。幅度受两条边界约束：
-- 上界 ShoreZMax 须远小于 DEFENSE_Z（走位下限 ShoreZ(x)+WALK_SHORE_MARGIN 不能顶到防线）、
-- 且 < BEACH_END_Z（BuildScenery 摆件落海自检断言）；下界别太靠海免得沙滩过宽。
Config.SHORE_PTS = {
    { x = -34.0, z = -1.0 },
    { x = -26.0, z =  2.2 },
    { x = -17.0, z = -1.6 },
    { x = -8.0,  z =  2.6 },
    { x =  0.0,  z = -0.6 },
    { x =  8.0,  z =  2.2 },
    { x = 17.0,  z = -1.6 },
    { x = 26.0,  z =  2.2 },
    { x = 34.0,  z = -1.0 },
}

--- 给定 x 处的水线 z（控制点间 Catmull-Rom 样条插值，区间外按端值平推）。
--- 过全部控制点、切线连续，拐角圆滑；几何与玩法都读这一个函数，曲线全链路一致。
---@param x number
---@return number
function Config.ShoreZ(x)
    local pts = Config.SHORE_PTS
```

```
    local n = #pts
    if x <= pts[1].x then
        return pts[1].z
    end
    if x >= pts[n].x then
        return pts[n].z
    end
    for i = 1, n - 1 do
        if x <= pts[i + 1].x then
            -- 本段 [p1,p2]，取前后邻居 p0/p3（端点处夹取自身）做 Catmull-Rom
            local p0 = pts[math.max(i - 1, 1)]
            local p1 = pts[i]
            local p2 = pts[i + 1]
            local p3 = pts[math.min(i + 2, n)]
            local t = (x - p1.x) / (p2.x - p1.x)
            local z0, z1, z2, z3 = p0.z, p1.z, p2.z, p3.z
            local t2 = t * t
            local t3 = t2 * t
            return 0.5 * ((2 * z1)
                + (-z0 + z2) * t
                + (2 * z0 - 5 * z1 + 4 * z2 - z3) * t2
                + (-z0 + 3 * z1 - 3 * z2 + z3) * t3)
        end
    end
    return pts[n].z
end

-- 样条在控制点之间可能有微小过冲，极值不能只扫控制点，得密集采样曲线本身，
-- 否则海直边/走位下限按“控制点极值”少留余量，过冲处会露出空洞或蓝面。
local SHORE_EXTREMA_SAMPLES = 64

--- 岸线的最靠海一处（下界）。海的矩形只能贴着它铺，否则曲线低谷里会露出蓝面。
---@return number
function Config.ShoreZMin()
    local pts = Config.SHORE_PTS
    local x0, x1 = pts[1].x, pts[#pts].x
    local minZ = math.huge
    for i = 0, SHORE_EXTREMA_SAMPLES do
        local z = Config.ShoreZ(x0 + (x1 - x0) * i / SHORE_EXTREMA_SAMPLES)
        if z < minZ then
            minZ = z
        end
    end
    return minZ
end

--- 岸线的最靠陆一处（上界）。所有“确保已上岸”的判据都按它算，不按平均值算。
---@return number
function Config.ShoreZMax()
    local pts = Config.SHORE_PTS
```

```
    local x0, x1 = pts[1].x, pts[#pts].x
    local maxZ = -math.huge
    for i = 0, SHORE_EXTREMA_SAMPLES do
        local z = Config.ShoreZ(x0 + (x1 - x0) * i / SHORE_EXTREMA_SAMPLES)
        if z > maxZ then
            maxZ = z
        end
    end
    return maxZ
end

-- 岸线的两个极值不另存常量：ShoreZMin()/ShoreZMax() 就是唯一算法，
-- 存一份副本只会多一个“改了控制点但忘了改它”的对账点。

-- 海面中心：+z 直边要探到“岸线最大值再往内陆一点”，水才漫得到那条曲线岸边的每一处
-- （否则曲线比 ShoreZMin 靠陆的列会出现“该有水却没水”的空洞）。沙滩网格已删，这条直线
-- 边不再由程序化沙盖住——它是否露出取决于场景里手摆的 patch-sand prefab 铺到哪。
-- （必须在 ShoreZMax 定义之后：文件级赋值时函数要是 nil，得到的是算术错误崩掉整个 Config）
Config.SEA_SHORE_TUCK = 0.4         -- 海边探过岸线最大值的量，保证曲线低谷列也有水覆盖
-- 海面整体朝内陆(+z)平移量：水线从岸线再往里推 100m（只平移、不扩海面，SEA_SIZE 不变）。
-- 注意：原本 +z 直边只 tuck 0.4 藏在沙底；这一推会让直边落到岸线以内 100m 处，
-- 水会漫过那一段沙滩（直边仍是直线，不再是曲线岸线）。
Config.SEA_INLAND_PUSH = 100.0
Config.SEA_CENTER_Z = (Config.ShoreZMax() + Config.SEA_SHORE_TUCK) - Config.SEA_SIZE * 0.5 +
Config.SEA_INLAND_PUSH

-- 沙滩网格（Coast.BuildSand）已删除，海岛地面改由场景手摆 patch-sand prefab 提供。
-- 下面只保留仍被玩法消费的量：BEACH_END_Z=装饰带内边界，SAND_Y=全局地面高度（Dog/Enemy/出生点读）。
Config.BEACH_END_Z = 28.0         -- 装饰带内边界：棕榈/岩石/点缀只摆在这条线以内（必须大于岸线最大值，见
Coast.BuildScenery 的自检）
Config.SAND_Y = 0.02                  -- 沙滩面高度，略高于海面以免共面闪烁
Config.DEFENSE_Z = 9.0                -- 防线：海盗走到这里就攻击，掉防线血

-- 主角阵地：四根 Dock_Pole 架一块木平台（不用 House2 当塔身——它 9.38m 高，
-- 站在 4m 的平台上会被整个包在屋子里看不见）。塔后另摆几栋木屋当村落剪影。
Config.TOWER_POS = Vector3(-12.0, 0.0, 15.0)
Config.PLATFORM_Y = 4.0               -- 木平台上表面高度，主角站在这
Config.PLATFORM_SIZE = Vector3(3.2, 0.22, 3.2)
Config.TOWER_POLE_INSET = 1.30        -- 四根腿相对平台中心的内缩量
-- Dock_Pole 本地 y 范围是 [-2.00, 1.01]，只缩 y（等比缩会让 0.36 的桩粗成 1.5m）：
-- 顶面要齐平台略高一点点当栏杆头，故 scaleY = (PLATFORM_Y + 0.15) / 1.01。
Config.TOWER_POLE_SCALE_Y = 4.11

Config.FOG_COLOR = Color(0.65, 0.74, 0.82)   -- 与 Sky.png 的地平线色 (166,188,208) 对齐
Config.FOG_START = 55.0
Config.FOG_END = 210.0
-- 环境光偏蓝（天光）而不是中性灰：沙子和木头受光面暖、背光面冷，画面才有早晚户外感。
-- 场景里原本写的是 0.35 0.35 0.4，这里接管过来，调光不用再去改 .scene。
Config.AMBIENT_COLOR = Color(0.32, 0.37, 0.46)
Config.SUN_DIRECTION = Vector3(0.45, -0.75, 0.28)
Config.SUN_COLOR = Color(1.0, 0.95, 0.86)    -- 略暖，海面才不像阴天
Config.SUN_BRIGHTNESS = 1.0
```

```
-- 天空盒：Skybox 组件 + CoreData 的 Box.mdl（已在抖音包里）。Skybox 的
-- ProcessRayQuery 是空实现，所以它永远不会挡住狙击射线，也不受远裁剪面影响。
Config.SKY_MODEL = "Models/Box.mdl"
Config.SKY_SCALE = 200.0

-- 布景件的落位（x, z, 绕 y 的角度，可选缩放）。数量与位置随意改，Coast.lua 只按表摆。
Config.PALMS = {
    { 6.5, 21.0, 20.0 }, { -7.5, 22.5, -35.0 }, { 13.0, 17.5, 70.0 },
    { -14.0, 19.0, 15.0 }, { 19.5, 11.0, -60.0 }, { -20.0, 12.5, 40.0 },
}
Config.ROCKS = {
    { 3.5, 5.0, 15.0, 1.0 }, { -5.0, 3.0, -40.0, 1.4 }, { 8.0, 7.5, 80.0, 0.9 },
    { -10.5, 6.0, 25.0, 1.2 }, { 15.0, 4.0, -15.0, 1.6 }, { -17.0, 8.0, 55.0, 1.1 },
    { 1.5, 1.5, 100.0, 0.8 }, { -2.5, 2.5, -70.0, 1.3 },
}
Config.CLIFFS = {
    { "Environment_Cliff1", -31.0, 9.0, 25.0, 1.0 },
    { "Environment_Cliff2", 33.0, 5.0, -40.0, 1.0 },
    { "Environment_Cliff4", -35.0, -8.0, 60.0, 1.0 },
    { "Environment_Cliff3", 30.0, 16.0, 15.0, 1.0 },
}
Config.DECOR = {
    { "Environment_Skulls", 11.0, 6.5, -20.0 },
    { "Environment_LargeBones", -18.0, 4.5, 35.0 },
    { "Prop_Anchor", 6.0, 2.0, 110.0 },
    { "Prop_Chest_Gold", -6.5, 7.0, -25.0 },
    -- Prop_Cannon 不在这里了：它是炮塔的炮身（见下方 TURRET 段）。同一个模型既是纯装饰
    -- 又能开炮就是"一个功能两个入口"，之前那颗还挂不到 noShoot、能被玩家打爆。
}
-- 村落：摆在塔的两侧而不是正后方。相机的偏航被限制在 ±62 度（朝着海），
-- 放在塔后（z > 19）的屋子根本进不了画面，白摆。
Config.VILLAGE = {
    { "Environment_House2", -16.5, 18.0, -25.0 },   -- 9.38m 的窄高楼，左侧剑影
    { "Environment_House1", 17.5, 20.5, 30.0 },     -- 13.5m 长屋，右侧
    { "Environment_House3", -24.0, 12.0, 60.0 },
}
-- 栈桥：Dock 模块沿 -z 排开，桥面落在 y≈1.0（模型自身 yMin=-2、高 3.0）
Config.DOCK_X = -8.5
Config.DOCK_Z_START = 1.6
Config.DOCK_STEP = 2.62
Config.DOCK_COUNT = 5

-- 可打爆的桶：前 3 个摆死，后 2 个是每条船甲板上带的（随船移动）
-- 栈桥那个的 y 是桥面高度：Dock 模块本地 y[-2.00, 1.01]，节点放在 SEA_Y 上，
-- 所以桥面 = SEA_Y + 1.01 = 0.81。
Config.BARREL_SPOTS = {
    { -8.5, 0.81, -3.6 },            -- 栈桥上
    { 4.5, 0.02, 6.5 },              -- 滩头
    { -3.5, 0.02, 8.5 },             -- 防线附近
```

```
}

-- ============================================================================
-- 滩头防线 = 两座会自己开炮的友方炮塔（取代原来那排 15 根木桩的栅栏）
--
-- 为什么布点是表而不是写死两颗：塔数/塔位改了，血量池（Waves）、索敌（Turrets）、
-- 视觉（Coast）都从这一张表出，不留第二份真相。下标即塔号。
-- 塔位 x=±9 配 RANGE=10 正好把 LANES 四条航道（±4.5 / ±13）全覆盖。
-- 台面刻意压成矮炮座而不是高塔：相机吊在主角后上方 35 度俯视朝海，塔一高就横切视线
-- （瞭望塔被挪到 x=-12 就是同一个原因）。
-- ============================================================================
Config.TURRETS = { -9.0, 9.0 }       -- 每座塔的 x，z 恒取 DEFENSE_Z
Config.TURRET_Z = Config.DEFENSE_Z
Config.TURRET_HP = 150               -- 每塔血量：两座合计 300 = 原 DEFENSE_HP，总抗漏量一格没变（难度基线不飘）
Config.TURRET_STAND_OFF = 1.6        -- 海盗砍塔时与塔中心的水平距离（停在台座外沿，不钻进模型里）
-- 炮台 = pirate-kit 瞭望塔 prefab（tower-complete-small）缩小一档坐地，塔顶再架一门 Prop_Cannon 当炮
-- （失能时 Turrets 把那根炮管歪下来）。塔身实测高 6.78m（解析 Cache 里 .mdl 顶点缓冲，同 Coast 头部
-- 那套方法），原尺寸会横切相机 35°俯视朝海的视线，所以按 Config.TURRET_TOWER_SCALE 缩到约 4.4m。
-- Prop_Cannon 实测盒 x[-0.52,0.52] y[0.0009,1.1217] z[-0.8665,0.6138]：炮管往 -z 伸得更长，即原生
-- 炮口就朝海，落位 yaw=0 不用转；原点在炮架底面，所以直接放塔顶高度即可。
Config.TURRET_TOWER_PREFAB = "tower-complete-small"
Config.TURRET_TOWER_SCALE = 0.65                              -- 6.78m 的塔缩一档 ≈4.4m，别挡视线
Config.TURRET_TOP_Y = 6.78 * Config.TURRET_TOWER_SCALE        -- 塔顶最高点（失能烟/爆闪、血条悬浮的基准）
-- 炮座：塔模型 6.78 是尖顶，能架炮的平台比它低一截，按塔顶放炮会悬空，故下压 0.2 坐进平台。
Config.TURRET_CANNON_SEAT_Y = Config.TURRET_TOP_Y - 0.2       -- 炮架底面落点（比塔顶低 0.2）
Config.TURRET_MUZZLE = Vector3(0.0, Config.TURRET_CANNON_SEAT_Y + 0.47, -0.85)  -- 炮口相对塔中心（炮座 + 炮身中段 0.47，朝海侧）
Config.TURRET_GUN_PITCH_DEAD = 35.0                -- 失能时炮管歪下去的角度，一眼看出这座塔已哑

-- 火控：定位是"漏防兜底"，且已收紧成只打海盗船——单发才 2 点，打 HP=1 的涌岸海盗是一炮一个、
-- 把玩家该割的草全抢了（用户反馈"海盗被完虐"），故只咬血厚的船（小船 3 / 大船 8），让它退回
-- "帮忙拆船、不抢人头"的位置。船停在海面 BOAT_STOP_Z、塔在 z=9，直线≈16m；旧的"上岸警戒线"
-- 那道 z 门槛本是给海盗设的、船永远在其外，所以删掉它，改成纯粹按 XZ 射程索敌，半径够到停船的航道。
-- 要再调炮台强弱只动 DAMAGE / INTERVAL 两个旋钮，别再把它放回打海盗。
Config.TURRET_RANGE = 40.0                         -- 对船 XZ 索敌半径（米）：船还在驶来途中就换炮（出生点 z=-44 离塔约 53m，仍够不到）
Config.TURRET_FIRST = 0.5                          -- 开局首发迟（秒）
Config.TURRET_INTERVAL = 4.0                       -- 两发间隔（秒）：小船两炮 8s、大船四炮 16s，兜底炮降一档
Config.TURRET_INTERVAL_SPREAD = 0.5                -- 间隔随机 ±，别让两塔齐射像节拍器
Config.TURRET_DAMAGE = 2                           -- 只咬船：小船 3 两炮、大船 8 四炮（对走路海盗已不出手）
Config.TURRET_SPLASH_RADIUS = 2.2
Config.TURRET_SHELL_SCALE = 0.65                   -- 弹丸大小：cannon-ball.prefab 原生≈0.64m，0.65 复刻旧弹≈0.42m 观感
Config.TURRET_FLIGHT_TIME = 0.45                   -- 飞行时长：短弧，友方射击不需要预警盘
Config.TURRET_LOB_HEIGHT = 2.6                     -- 抛物线拱高（米）
-- 开炮表现（纯演出，上面任何火控数值都不动）：炮管缓动转向 + 开炮后坐力回座。
Config.TURRET_AIM_TURN_RATE = 5.0     -- 转向趋近速率（指数）：半角程约 0.14s，快而不僵
Config.TURRET_AIM_FIRE_TOL = 15.0     -- 点火对准容差（度）：炮口没咬住目标先不开炮
Config.TURRET_RECOIL_AMPLITUDE = 0.3 -- 后坐力首退幅度（米）：炮管沿炮尾方向一挫
Config.TURRET_RECOIL_DECAY = 6.0      -- 晃动衰减率（1/秒）：一退一扑越晃越小
Config.TURRET_RECOIL_FREQ = 10.0      -- 晃动频率（弧度/秒）：回座的节拍
Config.TURRET_RECOIL_TIME = 1.0       -- 后坐力总时长（秒）：到点强制归位（此时幅度已衰减到不足 0.001，纯保险）
```

```
-------------------------------------------------------------------------
-- 相机（俯视角固定朝海，吊在主角正后上方；抖音小游戏刻意不需要玩家手动转视角）
--
-- 竖屏适配：水平视场角恒定（CAMERA_H_FOV）推出垂直 FOV，但竖屏被 VFOV_CAP 封顶；
-- 距离/俯角/前视按 t 从桌面值插值到 PORTRAIT 端值。竖屏端值的规格（可复解）：
--   * 人物在距屏幕底 1/5 屏高（中心下 0.3 屏高，占半屏 k=0.6）：
--     k = tan(pitch - atan(dist·sinP/(dist·cosP+lookAhead))) / tan(vfov/2)
--   * 人物占屏高 ≈11%（与桌面相当）：tan(1.8/dist) / (2·tan(vfov/2))
--   * 视线中心(lookAt)落在人物前方 lookAhead 处 ≈ 登陆带，画面上 4/5 全给海岸。
-- t=0 即桌面观感（1024x768 时垂直 FOV≈42，与改动前一致）。
-------------------------------------------------------------------------
Config.CAMERA_H_FOV = 54.0           -- 水平视场角（度），各端恒定
Config.CAMERA_DISTANCE = 20.0
Config.CAMERA_TARGET_Y = 2.85        -- 相机关注点相对主角脚底的高度
Config.CAMERA_PITCH = 35.0           -- 俯角（度），斜后方朝海面倾斜俯视
Config.CAMERA_LOOK_AHEAD = 6.0       -- 视线中心往海方向偏移，看到更大范围战场
Config.CAMERA_FAR_CLIP = 260.0
Config.CAMERA_KICK = 0.3             -- 开火相机微小震动幅度
Config.CAMERA_KICK_DECAY = 12.0
-- 竖屏端值与插值区间：aspect<=PORTRAIT_ASPECT 取满竖屏值，>=LANDSCAPE_ASPECT 用桌面原值
Config.CAMERA_PORTRAIT = { distance = 12.0, pitch = 30.0, lookAhead = 14.0 }
Config.CAMERA_PORTRAIT_ASPECT = 0.55
Config.CAMERA_LANDSCAPE_ASPECT = 1.3
Config.CAMERA_VFOV_CAP = 70.0        -- 垂直 FOV 上限，竖屏构图旋钮：96° 全视野人物只剩 2.4% 屏高；
                                     -- 70° 时人物 ≈11% 与桌面相当——横向覆盖收窄换主体清晰

--- 按画布宽高比（宽/高）派生本端相机参数。
---@param aspect number|nil 画布宽/高；nil 按桌面横屏
---@return { vfov: number, distance: number, pitch: number, lookAhead: number }
function Config.CameraForAspect(aspect)
    aspect = (aspect and aspect > 0.0) and aspect or Config.CAMERA_LANDSCAPE_ASPECT
    local t = (Config.CAMERA_LANDSCAPE_ASPECT - aspect)
        / (Config.CAMERA_LANDSCAPE_ASPECT - Config.CAMERA_PORTRAIT_ASPECT)
    t = math.min(math.max(t, 0.0), 1.0)
    local function lerp(a, b) return a + (b - a) * t end
    -- 水平恒定 ⇒ 垂直 FOV = 2·atan(tan(H/2)/aspect)
    local vfov = 2.0 * math.deg(math.atan(math.tan(math.rad(Config.CAMERA_H_FOV * 0.5)) / aspect))
    return {
        vfov = math.min(vfov, Config.CAMERA_VFOV_CAP),
        distance = lerp(Config.CAMERA_DISTANCE, Config.CAMERA_PORTRAIT.distance),
        pitch = lerp(Config.CAMERA_PITCH, Config.CAMERA_PORTRAIT.pitch),
        lookAhead = lerp(Config.CAMERA_LOOK_AHEAD, Config.CAMERA_PORTRAIT.lookAhead),
        -- 本端落在插值的哪一格（0=桌面原值，1=满竖屏端值）。调试面板显示它，否则在手机上
        -- 调“横屏端”那三行会毫无反应，看起来像面板坏了。
        t = t,
        aspect = aspect,
    }
end

-------------------------------------------------------------------------
```

```
-- UI 缩放（dp 方案）：rml 里全部用 dp 单位，1 dp = UI_SCALE 画布像素，
-- main.lua 启动时按画布短边算出并 rmlUi:SetScale 生效。基准短边 700px 时 1:1；
-- 小屏按比例缩（下限保可读/可点），大屏不放大（桌面观感与历史一致）。
-- 真机 393x852 → 393/700 ≈ 0.56。
-----------------------------------------------------------------------
Config.UI_DESIGN_SHORT_SIDE = 700.0
Config.UI_SCALE_MIN = 0.55
Config.UI_SCALE = 1.0   -- 占位：main.lua 启动时写入，Juice/摇杆几何读它

--- 按画布尺寸派生 UI 缩放系数。
---@param canvasW number
---@param canvasH number
---@return number
function Config.ComputeUIScale(canvasW, canvasH)
    local shortSide = math.min(canvasW or 0.0, canvasH or 0.0)
    if shortSide <= 0.0 then
        return 1.0
    end
    return math.min(math.max(shortSide / Config.UI_DESIGN_SHORT_SIDE, Config.UI_SCALE_MIN), 1.0)
end

-----------------------------------------------------------------------
-- 敌人
-----------------------------------------------------------------------
Config.LANES = { -13.0, -4.5, 4.5, 13.0 }
-- 船族现在全是 kenney prefab：小船/玩家船 ship-row-large、大船 ship-pirate-large
-- （旧的 Quaternius Ship_Small/Ship_Large 已连资产一起删除）。这批沿 z 建模，静置时船头
-- 朝海（= 退着开），故统一用 180 把船头拧向内陆（航行方向 +z）。玩家船驾船公式也以此为
-- 基准偏置（Player.lua），所以改这一处两侧同步生效；若某艘反而了，再按 kind 拆档。
Config.BOAT_YAW = 180.0
-- 船体视觉缩放，只作用于实例化出来的船体子节点（见 Enemy:SpawnBoat / Coast:BuildPlayerBoat），
-- 不缩逻辑节点：吃水、瞄点、坐船高度、沉船翻滚都是相对逻辑节点定义的。
Config.BOAT_SMALL_SCALE = 1.0        -- ship-row-large（小船 + 玩家船，实测 2.75 宽 x 0.85 高 x 2.85 长；旧船是 3.60 长，要更像旧体型可往上加）
Config.BOAT_BIG_SCALE = 0.8          -- ship-pirate-large（大船，真机量过：原尺寸过大、0.6 又偏小）
Config.BOAT_DRAFT = 0.15             -- 吃水：船体底面沉到水面下这么多，看着才像浮着
-- 船的 combat 瞄点抬高度（相对船 node 原点=船底）。船 node 在水面下，需抬到甲板附近，
-- 且保证命中点高于水面（SEA_Y）。数值按 .mdl 实测包围盒定：ship-row-large 高 0.85
-- （旧 Ship_Small 高 1.98，故旧值 1.2 已高于新船顶，子弹会打到船上方空气）。
Config.BOAT_HIT_HEIGHT = 0.45
-- 大船（ship-pirate-large）单独一档：原生高 9.96，×BOAT_BIG_SCALE=0.8 后船壳约 8m，
-- 与小船的 0.85 差一个数量级，共用会把大船的瞄点/炮口压到水线上。2.5 落在船壳中段。
Config.BOAT_BIG_HIT_HEIGHT = 2.5
Config.BOAT_SPAWN_Z = -44.0
Config.BOAT_STOP_Z = -6.5            -- 停船位置，海盗从这里下水
-- 船速/血：维持基线（不靠堆血量制造压力，已定案教训），只用略快的航速减少在射程里空转的时间。
Config.BOAT_SPEED = 4.2
Config.BOAT_BIG_SPEED = 3.0
Config.BOAT_UNLOAD_TIME = 0.55       -- 每吐一个海盗的间隔
Config.BOAT_HP = 3
Config.BOAT_BIG_HP = 8
```

```
Config.BOAT_CARRY = 7
Config.BOAT_BIG_CARRY = 12
Config.BOAT_SINK_TIME = 2.2          -- 沉船动画时长，之后移除节点

-- ------------------------------------------------------------------------
-- 大船炮台（仅 bigboat）：靠岸卸完兵后不立即离开，而是滞留往沙滩上抛炮弹。
-- 落点在主角可走的矩形内随机（“方向随机”=覆盖式炮击，不是瞄准主角），
-- 抛物线慢速 + 地面预警盘，给玩家看见并走位躲的时间；炸到主角掉大血。
-- ------------------------------------------------------------------------
Config.BOAT_CANNON_FIRST = 1.0           -- 停船后首发射迟（秒）
Config.BOAT_CANNON_INTERVAL = 3.0        -- 两发间隔（秒）
Config.BOAT_CANNON_INTERVAL_SPREAD = 0.8 -- 间隔随机 ±
Config.BOAT_CANNON_LINGER = 9.0          -- 卸完兵后大船滞留炮击的时长（秒），到点开走
Config.CANNON_BALL_SCALE = 0.85          -- 炮弹大小：cannon-ball.prefab 原生≈0.64m，0.85 复刻旧弹
≈0.55m 观感
Config.CANNON_FLIGHT_TIME = 2.2          -- 炮弹飞行时长（秒）：慢，留躲避时间
Config.CANNON_LOB_HEIGHT = 9.0           -- 抛物线拱高（米，峰在轨迹中点上方）
Config.CANNON_SPLASH_RADIUS = 3.2        -- 落点杀伤半径（米）
Config.CANNON_PLAYER_DAMAGE = 2          -- 炸到主角掉几格血（3 格口径：一发掉 2 格，与旧 100 血/30 伤
同比例）

Config.PIRATE_SPAWN_Z = -4.5         -- 下水点（浅水里，看着像涉水冲滩）
-- Quaternius 角色模型原生朝 -z（Soldier 的散落道具在 z 负侧就是"正前方"），
-- 海盗要朝 +z（防线/塔）行进，所以偏航 180 度。若进游戏发现海盗倒着走，改成 0。
Config.PIRATE_YAW = 180.0
Config.PIRATE_SPEED = 2.0            -- 调慢：海盗滩头滞留更久，给玩家更多“割”的时间与参与感
Config.PIRATE_SPEED_SPREAD = 0.5     -- 每个海盗速度随机 ±，避免排队齐步走
Config.PIRATE_HP = 1                 -- 一发死：保住割草爽感的根。压迫靠数量/来速，不靠肉。
-- 海盗 combat 瞄点抬高度（相对 node 原点=脚底）。角色模型原点在脚底，抬到躯干中段
-- 才能让子弹打身子、命中特效不喷在脚底。离屏形象缩到 0.5 倍后，胸口高度同比减半。
Config.PIRATE_HIT_HEIGHT = 0.45
Config.PIRATE_ATTACK_TIME = 1.1      -- 到防线后挥刀的时长，然后结算掉血
Config.PIRATE_DEATH_TIME = 1.6       -- 死亡动画播多久后移除
Config.PIRATE_HITREACT_TIME = 0.28
Config.PIRATE_WADE_DEPTH = 0.35      -- 涉水时把模型抬高一点，别半截埋在水里
-- 突袭型海盗：不按航道直冲防线，而是转追主角、近身后反复砍主角（走位压力的来源）。
-- 生成时按概率标记，用稀有皮（骷髅/船长）做视觉区分；hp 更肉一点避免一发带走。
Config.PIRATE_RAIDER_CHANCE = 0.6    -- 每个海盗是突袭型（追主角）的概率。分流阀：调高→追你的多、冲海岸的
自动少（总数不变）
Config.PIRATE_RAIDER_HP = 3          -- 突袭型血量（普通型走 PIRATE_HP=1）：追身的略肉一点避免一发带走
Config.PIRATE_MELEE_RANGE = 1.7      -- 与主角水平距离小于此值即停下挥刀
Config.PIRATE_PLAYER_DAMAGE = 1      -- 突袭型一刀砍主角几格血（3 格口径：一刀掉 1 格）
Config.MAX_ALIVE_UNITS = 32          -- 同屏单位闸：超了就延后生成。船能整船靠岸后上岸量猛增，拉到 32 保
证潮水不断流（真机卡就回调）
Config.FAR_UNIT_DISTANCE = 55.0      -- 超过这个距离的单位关掉不可见时也更新

-- 性能 Overlay（调试开关）：true 时屏幕顶中叠加一行 FPS/drawcall/三角面/同屏单位数（每 3 秒刷）。
-- 上线前必须置 false（关掉就不显不刷）。真机（抖音）没有引擎 DebugHud（SystemUI 已禁用），靠这个看指标。
Config.PERF_OVERLAY = false

-- 相机参数调试面板（调试开关）：true 时 HUD 右上角多一颗“相机”按钮，点开可在真机上实时拖
-- 本节全部相机常量并打印可直接粘回本文件的数值（见 game/CamDebug.lua + UI/CamDebug.rml）。
-- 上线前必须置 false：为 false 时 main.lua 根本不创建那个 RmlUI 组件，按钮与面板都不存在，
-- 不是“隐藏了”而是没有这份 UI。
```

```
Config.DEBUG_CAM_PANEL = false

-- 帧率上限（省电/降热）。真机（抖音 wasm）由引擎主循环按此值节流；桌面/编辑器同步生效。
-- 0 或 nil = 不改引擎默认（wasm 默认 30）。改这个值只需重打包（不碰 wasm），main.lua 启动时
-- 经 engine:SetWebFrameRate(Config.MAX_FPS) 落地。
Config.MAX_FPS = 40

-------------------------------------------------------------------------
-- 广告（休眠链路）：激励视频「死亡复活」。引擎侧 ShowRewardedAd 已全通
-- （C++ → Lua 绑定 → JS handler，看完整判定 isEnded），游戏侧只在这里取广告位 ID。
--
-- 休眠语义：REWARDED_UNIT_ID 为空字符串时，结算面板的复活按钮整条不渲染
-- （data-if 链路零暴露，玩家看不到任何广告痕迹）——当前状态。
-- 拿到流量主广告位后的启用步骤：填入 adUnitId → 重打包 → 抖音后台提交快审更新
-- （1-3 天），无需重编 wasm、无需改其它代码。
-- 官方口径：仅线上状态的小游戏调用广告才产生收益，未上线版本埋广告代码不违规。
--
-- 远程覆盖预留位：将来有 HTTPS 备案域名后，可在此处加一个远程 JSON 拉取，
-- 读到的远程 adUnitId 覆盖本地常量（读取收口在 main.lua 一处，改 adUnitId 即免重审）。
-- 现在只留本地常量，不做任何网络请求。
-------------------------------------------------------------------------
Config.AD = {
    REWARDED_UNIT_ID = "",          -- 抖音流量主激励视频广告位 ID；空 = 复活按钮不显示
}

-------------------------------------------------------------------------
-- 侧边栏复访（抖音必接合规）：从侧边栏进入的老用户给一档会话增益作回归奖励；
-- 非侧边栏首次进入弹一次引导（去侧边栏 / 知道了，见 game/Sidebar.lua）。
-- 增益数值要显性可感知：攻击快、活得久——审核与玩家两头都好交代。
-- 会话级生效：每次从侧边栏冷启动/回前台的游戏会话都带，提示只发一次防来回刷。
-------------------------------------------------------------------------
Config.SIDEBAR_DMG_MULT = 1.2       -- 侧边栏回归：每发子弹伤害倍率（主角自动开火那一条路）
Config.SIDEBAR_HP_MULT = 1.25       -- 侧边栏回归：主角生命上限倍率（Waves.playerMaxHp）
-- 调试模拟（编辑器里没有宿主 SDK，真机链路走不到）："reward" 模拟侧边栏进入发增益，
-- "guide" 模拟首次非侧边栏进入弹引导，false = 上线值走真实 SDK。
Config.DEBUG_SIDEBAR = false

Config.PIRATE_SKINS = { "Characters_Anne", "Characters_Henry", "Characters_Mako",
"Characters_Sharky" }
Config.PIRATE_SKIN_RARE = { "Characters_Skeleton", "Characters_Captain_Barbarossa" }

-------------------------------------------------------------------------
-- 武器（全自动开火：射程内取最近敌人自动射击，玩家只管走位——抖音小游戏的刻意简化）
-- 注：没有手动射线/准星/磁吸/爆头——这些都是早期“手动狙击”设想留下的残留配置，已随之清除。
-------------------------------------------------------------------------
Config.MAG_SIZE = 5
Config.RELOAD_TIME = 1.2
Config.FIRE_COOLDOWN = 0.22          -- 两发之间的最小间隔（不降扳机频率，总伤害靠穿透/爆炸来收）
Config.FIRE_WINDOW = 0.6             -- 开火后维持射击姿态/上半身对准目标的窗口（秒）：大于冷却
                                     -- 保证连发期间不断档，装弹期间自然回落举枪
Config.ATTACK_RANGE = 45.0           -- 自动吸附半径：主角周围这个半径内取最近敌人开火（无需手动对准）
```

```
-- ------------------------------------------------------------------------
-- 扇形散射 + 局内成长（割草爽感的两个相乘因子：命中广度 × 力量可见增长）。
-- 一次扳机扣 1 发弹药（弹匣是“扳机数”不是“弹丸数”），沿 SCATTER_SPREAD_DEG 扇形
-- 喷出 pellets 颗；子弹改为沿线扫掠命中（撞到航道上第一个敌人即结算并消耗），
-- 所以散射 = 一次清一条航道若干人，多点同时炸。pellets/damage 由 game/Grow.lua
-- 按 GROWTH 曲线随击杀经验爬升。
-- ------------------------------------------------------------------------
Config.SCATTER_SPREAD_DEG = 26.0      -- 扇形总张角（度，多弹沿此弧均匀铺开）
Config.BULLET_HIT_RADIUS = 1.3        -- 子弹扫掠命中半径（米）
-- 曳光弹造型：Box.mdl（12 三角面）拉成细长亮芯 + 加色发光材质（Materials/BulletTracer.xml），
-- 替换旧的 54KB Sphere.mdl 实心球；微火花拖尾是常驻发射器，随弹销毁，false 可整关。
Config.BULLET_BOLT_THICK = 0.055      -- 亮芯截面边长（米）
Config.BULLET_BOLT_LEN = 0.85         -- 亮芯长度（米）
Config.BULLET_TRAIL = true            -- 子弹微火花拖尾（每颗弹 1 个 draw call，可关）
Config.GROW_XP = { pirate = 1, boat = 6, bigboat = 20, barrel = 2 }  -- 各类击杀喂的经验
-- 成长曲线：每项 { xp=达到该级所需累计经验，pellets=扇形弹丸数，damage=单发伤害，pierce=子弹可贯穿的单位数 }。
-- 首项是开局形态（xp 恒 0）；到达某级即采用该项，之后只增不减。pierce=1 即一发只杀一个、命中即消亡；
-- >1 时一颗子弹能沿航道串穿一排单位（路 A 后已全线收回 1，若日后要做“后期穿透奖励”再单独抬顶档）。
-- 经验阈值按加密后的三波总击杀量标定，使成长贯穿全局而不早早封顶（真机体感不对就改 xp 间距）。
-- pellets 永远取奇数（当前表 1/3/5/7）：扇形按 spread*(i-0.5)/pellets 均匀铺开，只有奇数发才
-- 有一颗处于 a=0° 的正中弹直射锁定目标。偶数发（如 2）正中是空的，两颗对称夹住
-- 小船（船体仅 ~1.3m 宽）会刚好从缝里漏掉——故不用偶数。
Config.GROWTH = {
    { xp = 0,    pellets = 1, damage = 1, pierce = 1 },
    { xp = 50,   pellets = 3, damage = 1, pierce = 1 },
    { xp = 140,  pellets = 3, damage = 1, pierce = 1 },
    { xp = 300,  pellets = 5, damage = 1, pierce = 1 },   -- 路 A：pellets≥5 段取消穿透（穿 2→1），不再一梭贯穿整列纵队
    { xp = 550,  pellets = 5, damage = 1, pierce = 1 },   -- 路 A：同上，单扣命中上限 = 弹丸数（5）
    { xp = 900,  pellets = 5, damage = 1, pierce = 1 },   -- 路 A：穿 2→1（A1 削顶仍留 2 穿，实测仍打爆，故连穿透一起收）
    { xp = 1400, pellets = 7, damage = 1, pierce = 1 },   -- 路 A：穿 2→1，扇形 7 弹但每发只杀 1，保住散射爽感不刮穿
    { xp = 2100, pellets = 7, damage = 2, pierce = 1 },   -- 路 A：穿 2→1，满级单扣命中上限 14→7
}

-- ------------------------------------------------------------------------
-- 打击感三件套（hit-stop / 震屏 / 飘字）：统一由 game/Juice.lua 驱动，
-- 触发点集中在 main.lua 的 OnScore（玩家子弹与爆桶连锁都汇入那里）。
-- ------------------------------------------------------------------------
Config.HITSTOP_SCALE = 0.05       -- 击杀定格期间的世界时间流速（5%：近停不停，
                                  -- 全停会让连续击杀变成卡顿感）
Config.SHAKE_DECAY = 2.8          -- 震屏强度（trauma）每秒线性衰减量
Config.SHAKE_AMPLITUDE = 0.55     -- 相机位置抖动幅度（米，trauma=1 峰值时）
Config.SHAKE_ROLL = 2.5           -- 相机横滚抖动幅度（度，trauma=1 峰值时；
                                  -- 俯视角下小滚角让“海面倾斜”，比纯平移震更带感）
Config.POPSCORE_LIFE = 0.9        -- 飘字寿命（秒）
Config.POPSCORE_RISE = 90.0       -- 飘字上浮速度（画布像素/秒，随寿命减速到 30%）

-- ------------------------------------------------------------------------
-- 自定义语音（麦克风录音→宿主侧存 aac→变声回放）：全链路在 MinigameSDK/
-- minigame_sdk.js（引擎只发请求收事件，不碰音频解码）。格式只能是宿主录音器
```

```
-- 收的 aac/pcm/wav（抖音实测 mp3 被拒），音色 playbackRate 必须
-- 取宿主支持的档位（0.5/0.8/1.0/1.25/1.5/2.0），非档位值会被平台钳到最近档。
-------------------------------------------------------------------------
Config.VOICE_MAX_SEC = 3          -- 单句录音上限（秒），到时自动停
Config.VOICE_FILTERS = {
    { name = "海盗低音", rate = 0.8 },
    { name = "原声",     rate = 1.0 },
    { name = "哈士奇",   rate = 1.5 },
}
Config.VOICE_DEFAULT_FILTER = 2   -- 默认原声（VOICES_FILTERS 下标）
-- 槽位：宿主侧存为 rbfx_voice_<slot>.aac，Lua 与 JS 共享的唯一契约
Config.VOICE_SLOT_LOSE = "lose"        -- 防线失守时播
Config.VOICE_SLOT_KILLSTREAK = "killstreak"  -- 连杀里程碑播（阈值同 COMBO_VOICE_AT）

-------------------------------------------------------------------------
-- 移动（左摇杆走位，相机相对方向；抖音包无物理，靠矩形边界钳制，不做碰撞）
-------------------------------------------------------------------------
Config.MOVE_SPEED = 4.2              -- 米/秒
Config.TURN_SPEED = 8.0              -- 人物转向速度（弧度/秒），朝移动方向转
Config.AIM_MAX_TWIST = 1.75          -- 上半身相对身体朝向的最大扭转（弧度，约 100 度）
Config.AIM_TURN_SPEED = 12.0         -- 上半身扭转的平滑速度（弧度/秒）

-- 右手扶枪（程序化 IK）：Characters_Lis 的 _Gun 动画（Idle_Gun/Run_Gun）里右臂是自然下垂的
-- （这个姿态烘在 FBX 资片里，不是接线引入的）。这里在自定义动画时点（ApplyAim→GunHold）把
-- 右臂两节骨骼 UpperArm.R/LowerArm.R 解析地摆到步枪握把（左手中指骨 Middle1.L，步枪的挂点）。
-- 调参全靠目视：手不到位→加大 strength；肘朝外/穿模→改 pole 侧或调 gripOffset；拖动→降 ramp。
Config.GUN_HOLD = {
    enable = true,
    strength = 0.9,        -- 0=完全跟随动画(手垂着)，1=手精确贴到枪上
    ramp = 12.0,           -- 进/出枪态时权重淡入淡出速度（每秒）
    minBend = 0.10,        -- 肘部最小余量(占满臂长比例)，防完全伸直奇点
    gripOffset = Vector3(0.0, 0.0, 0.0),  -- 相对 Middle1.L 的世界系握把目标微调
}
Config.PLAYER_SPAWN = Vector3(0.0, Config.SAND_Y, 11.0)   -- 脚底出生点：防线前一点的地面，朝向海
-- 主角模型整体放大倍数（对 Player 节点 SetScale）。模型/骨骼都在脚底原点之下，以脚为锚
-- 往上长，不影响出生点与走位；GunHold 的 IK 读骨骼世界坐标（同比缩放）故仍自洽。
Config.PLAYER_MODEL_SCALE = 1.5
Config.WALK_MIN_X = -18.0
Config.WALK_MAX_X = 18.0
-- 走位下界不是一个常量，而是“那个 x 处的岸线 + 余量”（见 Player:Move）：岸线是曲线，
-- 写死一个 z 就会出现“在湾里已经踩进水里、判据还说你在沙上”。余量沿旧口径：旧版把
-- 下界写死在 1.5，配直线水线 -0.5 等于留 2.0m，这里保持同一个 2.0m。
Config.WALK_SHORE_MARGIN = 2.0       -- 别走进水里：相对岸线的余量
Config.WALK_MAX_Z = 22.0             -- 别绕到塔后村子里

-- 宠物狗（见 game/Dog.lua）：友方伴随单位，独立于 enemy.units（玩家子弹扫不到它、波次
-- 清怪不含它、爆桶连锁不误伤它）。平时在主角身边溜达，视野内有海盗就冲上去咬（静默击杀，
-- 不进 onScore）；海盗不主动攻击狗；只有大船炮弹能把狗炸飞上天翻滚，狗不死（无血量）。
Config.DOG = {
    spawnOffset = Vector3(1.6, 0.0, 1.2),  -- 出生/重开时相对主角脚底的偏移
```

```
    speed = 4.0,           -- 追击/回家的移动速度（米/秒，须快过海盗 2.3~2.6 才追得上）
    wanderSpeed = 1.7,     -- 身边溜达的速度
    followRadius = 6.0,    -- 离主角超过这个距离就先跑回身边
    followStopRadius = 2.5, -- “跑回身边”的迟滞：一旦启动回家，就一直跑到距主角小于此值才改回溜达（防在
followRadius 边界逐帧横跳→逐帧朝内/朝外翻面）
    wanderRadius = 4.0,    -- 在主角周围这么大范围内随机乱逛
    wanderMinDist = 1.5,   -- 重选漫游目标点时、新点需离狗至少这么远（否则走两步即“到达”→几乎立刻翻面重走）
    wanderArrive = 0.5,    -- 距漫游目标点小于此值判定到达，原地待机到计时结束再挑下一个
    aggroRadius = 11.0,    -- 看到这个半径内的存活海盗就去咬
    biteRange = 1.1,       -- 与海盗水平距离小于此值即下口
    faceMinDist = 0.35,    -- 与目标水平距离小于此值时不更新朝向（近距 atan 输入是亚分米噪声向量，逐帧会甩
头）
    faceTurnRate = 540.0,  -- 朝向最大角速度（度/秒）：把“瞬时翻面”摊成有限速的平滑转身，边界跳变不再逐帧
反向
    faceTurnEps = 4.0,     -- 转向死区（度）：已对准目标时不再反复微调，消除对准后的微幅摆抖
    stickFactor = 0.7,     -- 目标粘滞：当前锁定的目标距离×此系数 ≤ 新最佳距离时不切换，防等距 A↔B 横跳
    biteDamage = 2,        -- 每口伤害（普通海盗 HP=1 一口秒，突袭型 HP=3 需两口）
    biteInterval = 0.7,    -- 两次下口的间隔（秒）
    biteAnim = "Attack",   -- 咬击动画栈名
    barkCooldown = 1.1,     -- 追击/攻击途中两次吠叫的最小间隔（秒）
    restDuration = 10.0,    -- 一波清完后狗原地趴卧休息的时长（秒），期间播喘气、不移动不索敌
    restArriveRadius = 2.5, -- 波次清完“跑回主角身边”后、判定已到位可休息的水平半径（米）
    -- 被炮弹炸飞：纯 Lua 抛物线积分 + 多轴翻滚（抖音包无物理，绝不做刚体）
    launch = {
        upSpeed = 14.0,      -- 初始上升速度（米/秒）
        gravity = 20.0,      -- 回落加速度（米/秒²）：比真重力大，飞得起落得快、节奏紧凑
        spinSpeed = 720.0,   -- 翻滚角速度（度/秒），绕斜轴拧着翻
        forwardKick = 3.2,   -- 沿“爆炸→狗”水平方向的弹出初速（米/秒），带阻尼衰减
        recoverTime = 0.6,   -- 落地后滚回四脚着地的起身时长（秒）
    },
}

--------------------------------------------------------------------------
-- 载具（玩家专属船：靠近码头按交互键登船，摇杆改驱动船在海域自由探索）
--------------------------------------------------------------------------
Config.BOAT_SPAWN = Vector3(-6.0, Config.SEA_Y, 5.0)   -- 船停在码头旁水面上
-- 角色坐船的局部高度（相对船节点）。坐太低会被船舷从俯视相机角度看进“舱里”看不见，故抬到甲板
-- 附近。新船 ship-row-large 实测高 0.85（y[0,0.85]，旧 Ship_Small 是 1.98），主角髋部大约在甲板。
Config.BOAT_SEAT_Y = 0.45
-- 船上不再设固定炮位：人与站在地面一致，从枪口（枪骨下的 ShootingPos）出弹。
-- 注意：不要复用 Config.BOAT_SPEED（那是敌人船速，见 Enemy.lua），玩家船速单独命名
Config.PLAYER_BOAT_SPEED = 8.0       -- 米/秒，比走路快
Config.PLAYER_BOAT_TURN = 3.0        -- 船转向速率（弧度/秒），比人物慢，做出惯性
Config.BOAT_INTERACT_DIST = 5.0      -- 角色与船水平距离小于此值可登船
-- 海域可行走矩形（驾船时）：比沙滩大得多，但不能开上沙滩
Config.SEA_MIN_X = -60.0
Config.SEA_MAX_X = 60.0
Config.SEA_MIN_Z = -80.0             -- 往海方向能开很远
-- 靠岸线按“岸线最靠陆那处 + 余量”算：按平均值或按某个 x 算，船会在两个湾里搁浅。
Config.SEA_MAX_Z = Config.ShoreZMax() + 3.0   -- 靠岸余量（下船不限制位置，随时可下）

--------------------------------------------------------------------------
-- 计分、连击、防线
```

```
---@param id string
function Hud:HideButton(id)
    if id == "board" then
        if self.showBoard then
            self.showBoard = false
            self:Dirty("showBoard")
        end
    elseif id == "disembark" then
        if self.showDisembark then
            self.showDisembark = false
            self:Dirty("showDisembark")
        end
    end
end

-- ============================================================================
-- 局内成长（game/Grow.lua 推）
-- ============================================================================

--- 等级数字。
function Hud:SetLevel(v)
    self.level = v or 1
    self:Dirty("level")
end

--- 经验条进度 0..1（当前级到下一级的百分比）。
function Hud:SetXpProgress(v)
    self.xpProg = clamp01(v or 0.0)
    self:Dirty("xp")
end

--- 武器文本（“弹丸 xN”），升级时随当前弹丸数刷新。
function Hud:SetWeaponText(text)
    self.weaponText = text or ""
    self:Dirty("weaponText")
end

-- ============================================================================
-- 自定义语音（game/Voice.lua）
-- ============================================================================

--- 是否显示语音入口（宿主 SDK 可用才显示；编辑器/桌面整块不渲染）。
function Hud:SetVoiceAvailable(b)
    self.voiceAvailable = b and true or false
    self:Dirty("voiceAvailable")
end

--- 打开语音面板（模态，main.lua 据此暂停世界推进）。
function Hud:OpenVoice()
```

```
    if not self.voiceOpen then
        self.voiceOpen = true
        self:Dirty("voiceOpen")
    end
end

function Hud:CloseVoice()
    if self.voiceOpen then
        self.voiceOpen = false
        self:Dirty("voiceOpen")
    end
end

---@return boolean 语音面板是否开着（帧循环用它把世界 dt 压成 0）
function Hud:IsVoiceModal()
    return self.voiceOpen
end

--- 录音中标记（面板上亮红点）。
function Hud:SetVoiceRecording(b)
    self.voiceRecording = b and true or false
    self:Dirty("voiceRecording")
end

--- 面板状态行（录好了 / 没授权麦克风 / 还没录过）。
---@param text string
function Hud:SetVoiceStatus(text)
    self.voiceStatus = text or ""
    self:Dirty("voiceStatus")
end

--- 当前变声音色名（Voice:CycleFilter 的返回值）。
---@param name string
function Hud:SetVoiceFilterName(name)
    self.voiceFilterName = name or ""
    self:Dirty("voiceFilterName")
end

-------------------------------------------------------------------------
-- 侧边栏复访引导（game/Sidebar.lua）
-------------------------------------------------------------------------

--- 打开引导弹窗（模态，main.lua 据此暂停世界推进）。
function Hud:OpenSidebarGuide()
    if not self.sidebarGuide then
        self.sidebarGuide = true
        self:Dirty("sidebarGuide")
    end
end
```

```
function Hud:CloseSidebarGuide()
    if self.sidebarGuide then
        self.sidebarGuide = false
        self:Dirty("sidebarGuide")
    end
end

---@return boolean 引导弹窗是否开着（帧循环用它把世界 dt 压成 0）
function Hud:IsSidebarGuideModal()
    return self.sidebarGuide
end

--- 每帧：hitmarker 到点自动关闭 + 旁白到期清除。
---@param dt number
function Hud:Update(dt)
    if self.hitT > 0.0 then
        self.hitT = self.hitT - dt
        if self.hitT <= 0.0 then
            self.hitT = 0.0
            if self.hitNormal or self.hitStrong then
                self.hitNormal = false
                self.hitStrong = false
                self:Dirty("hitNormal")
                self:Dirty("hitStrong")
            end
        end
    end
    if self.narrateT > 0.0 then
        self.narrateT = self.narrateT - dt
        if self.narrateT <= 0.0 and self.narrate ~= "" then
            self.narrate = ""
            self:Dirty("narrate")
        end
    end
end

return Hud
-- Scripts/game/CamDebug.lua
--
-- 相机参数真机调试面板：一颗"相机"按钮 + 一块滑条面板，实时改 game/Config.lua 里的相机
-- 常量并立刻反映到画面。只在 Config.DEBUG_CAM_PANEL=true 时被 main.lua 创建——上线态下这份
-- UI 根本不存在（不是隐藏）。
--
-- 机制（照 107_HelloRmlUI 与 Hud.lua 的同一套）：
--   * SetHandlers{onInit} 必须先于 SetResource；onInit 是唯一能 Bind* 的时机。
--   * 滑条走 BindProperty（读写双向）：RML 里 <input type="range" data-value="name">，rbfx 的
--     DataControllerValue 监听元素 change 事件把新值写回 model ⇒ 落到这里的 setter（官方样例
--     的 slider_value 就是这个用法）。rmlUi 传回的是字符串，故 tonumber 兜一层。
--   * 每行的数值文本另开一个只读绑定 name.."Text"：数字要带单位/精度，且绝不在内联 style 里
--     绑 {{}}（RCSS 解析器会崩），所以格式化后的字符串单独喂给 <td>。
```

```
--    * 拖动过程中不回写 data-value（不 Dirty 滑条自己的绑定名），否则 model→element 方向会把
--      正在拖的把手按旧值重设，手感变成"拖不动"。
--
-- 改参数后为什么还要 Player:ApplyCameraFit()：距离/俯角/前视是 Player 每帧读的缓存成员，
-- FOV/远裁剪面是写死在相机组件上的，两者都不会自己感知 Config 变了。
--
-- "打印"是这条调试链的出口：手机上看不到 Config.lua，调好的值以可粘贴的 Lua 语句打到日志，
-- 回桌面照抄即可。

local Config = require("game.Config")

---@class CamDebug 相机参数调试面板
local CamDebug = {}
CamDebug.__index = CamDebug

--- 可调参数表。RML 的绑定名必须与 key 一一对应（UI/CamDebug.rml 的 data-value 与 {{...Text}}）。
--- src 是"打印"按钮输出时对应的 Config 写法。
local PARAMS = {
    -- 竖屏端：真机（抖音竖屏 t=1）实际生效的就是这三行
    { key = "distPort", src = "Config.CAMERA_PORTRAIT.distance", fmt = "%.2f",
      get = function() return Config.CAMERA_PORTRAIT.distance end,
      set = function(v) Config.CAMERA_PORTRAIT.distance = v end },
    { key = "pitchPort", src = "Config.CAMERA_PORTRAIT.pitch", fmt = "%.1f",
      get = function() return Config.CAMERA_PORTRAIT.pitch end,
      set = function(v) Config.CAMERA_PORTRAIT.pitch = v end },
    { key = "lookPort", src = "Config.CAMERA_PORTRAIT.lookAhead", fmt = "%.2f",
      get = function() return Config.CAMERA_PORTRAIT.lookAhead end,
      set = function(v) Config.CAMERA_PORTRAIT.lookAhead = v end },

    -- 横屏端：t=0 时生效（编辑器 GameView / 桌面）
    { key = "distLand", src = "Config.CAMERA_DISTANCE", fmt = "%.2f",
      get = function() return Config.CAMERA_DISTANCE end,
      set = function(v) Config.CAMERA_DISTANCE = v end },
    { key = "pitchLand", src = "Config.CAMERA_PITCH", fmt = "%.1f",
      get = function() return Config.CAMERA_PITCH end,
      set = function(v) Config.CAMERA_PITCH = v end },
    { key = "lookLand", src = "Config.CAMERA_LOOK_AHEAD", fmt = "%.2f",
      get = function() return Config.CAMERA_LOOK_AHEAD end,
      set = function(v) Config.CAMERA_LOOK_AHEAD = v end },

    -- 视场
    { key = "fovH", src = "Config.CAMERA_H_FOV", fmt = "%.1f",
      get = function() return Config.CAMERA_H_FOV end,
      set = function(v) Config.CAMERA_H_FOV = v end },
    { key = "vfovCap", src = "Config.CAMERA_VFOV_CAP", fmt = "%.1f",
      get = function() return Config.CAMERA_VFOV_CAP end,
      set = function(v) Config.CAMERA_VFOV_CAP = v end },

    -- 取景与手感
    { key = "targetY", src = "Config.CAMERA_TARGET_Y", fmt = "%.2f",
```

```
      get = function() return Config.CAMERA_TARGET_Y end,
      set = function(v) Config.CAMERA_TARGET_Y = v end },
    { key = "farClip", src = "Config.CAMERA_FAR_CLIP", fmt = "%.0f",
      get = function() return Config.CAMERA_FAR_CLIP end,
      set = function(v) Config.CAMERA_FAR_CLIP = v end },
    { key = "kick", src = "Config.CAMERA_KICK", fmt = "%.2f",
      get = function() return Config.CAMERA_KICK end,
      set = function(v) Config.CAMERA_KICK = v end },
    { key = "kickDecay", src = "Config.CAMERA_KICK_DECAY", fmt = "%.1f",
      get = function() return Config.CAMERA_KICK_DECAY end,
      set = function(v) Config.CAMERA_KICK_DECAY = v end },

    -- 插值区间端点（动它们等于移动 t 本身，故单列一组，避免和端值行混在一起看不出因果）
    { key = "aspPort", src = "Config.CAMERA_PORTRAIT_ASPECT", fmt = "%.2f",
      get = function() return Config.CAMERA_PORTRAIT_ASPECT end,
      set = function(v) Config.CAMERA_PORTRAIT_ASPECT = v end },
    { key = "aspLand", src = "Config.CAMERA_LANDSCAPE_ASPECT", fmt = "%.2f",
      get = function() return Config.CAMERA_LANDSCAPE_ASPECT end,
      set = function(v) Config.CAMERA_LANDSCAPE_ASPECT = v end },
}

--- 当前画布宽高比。问 RmlUI 要 backbuffer 尺寸（与 main.lua 的 CanvasSize 同一来源：
--- 编辑器 GameView 下 Graphics:GetWidth 是整个编辑器窗口，只有 RmlUi 的 canvas 才对得上画面）。
---@return number|nil
local function CanvasAspect()
    local ui = GetSubsystem("RmlUI")
    if ui and ui.GetCanvasSize then
        local size = ui:GetCanvasSize()
        if size.y > 0 then
            return size.x / size.y
        end
    end
    return nil
end

---@param scene Scene
---@param player Player 被调参的相机宿主（Player:ApplyCameraFit 落回相机）
---@return CamDebug
function CamDebug.Build(scene, player)
    local self = setmetatable({}, CamDebug)
    self.player = player
    self.showTrigger = true
    self.showPanel = false
    self.statusLine = ""
    self.statusLine2 = ""
    self.statusT_ = 0.0

    -- 复位基准：取面板创建那一刻的值（即 Config.lua 里的出厂值）。
    -- 存快照而不是把默认值再抄一遍，避免两处默认值随时间走偏。
    self.defaults = {}
```

```
for _, p in ipairs(PARAMS) do
    self.defaults[p.key] = p.get()
end

local comp = scene:CreateComponent("LuaRmlUIComponent")
self.comp = comp

comp:SetHandlers({
    onInit = function(w)
        self.w = w
        w:BindReadonlyProperty("showTrigger", function() return self.showTrigger end)
        w:BindReadonlyProperty("showPanel", function() return self.showPanel end)
        w:BindReadonlyProperty("statusLine", function() return self.statusLine end)
        w:BindReadonlyProperty("statusLine2", function() return self.statusLine2 end)

        for _, p in ipairs(PARAMS) do
            local textKey = p.key .. "Text"
            -- 只读那一份是格式化文本；读写那份是滑条本身
            w:BindReadonlyProperty(textKey, function()
                return string.format(p.fmt, p.get())
            end)
            w:BindProperty(p.key,
                function() return p.get() end,
                function(v)
                    local n = tonumber(v)
                    if n then
                        p.set(n)
                        self:Apply(p.key)
                    end
                end)
        end

        w:BindEvent("open", function() self:SetOpen(true) end)
        w:BindEvent("close", function() self:SetOpen(false) end)
        w:BindEvent("reset", function() self:Reset() end)
        w:BindEvent("dump", function() self:Dump() end)
    end,
    -- 每秒刷一次状态行（含 FPS）：面板只在这期间占屏，每帧 MarkDirty 是白付的开销。
    update = function(_, dt)
        if not self.showPanel then
            return
        end
        self.statusT_ = self.statusT_ + (dt or 0.0)
        if self.statusT_ >= 1.0 then
            self.statusT_ = 0.0
            self:RefreshStatus()
        end
    end,
})
```

```
    comp:SetResource("UI/CamDebug.rml")
    self:RefreshStatus()
    return self
end

---@private
function CamDebug:Dirty(name)
    if self.w then
        self.w:MarkDirty(name)
    end
end

--- 参数变了：重跑相机适配 + 刷面板文本。
--- skipKey 是刚被拖动的那个滑条——它的 data-value 不回写（见文件头），其余滑条要刷，
--- 因为改插值区间端点会连带移动 t，别的行可能确实换了生效值。
---@param skipKey string|nil
function CamDebug:Apply(skipKey)
    if self.player and self.player.ApplyCameraFit then
        self.player:ApplyCameraFit()
    end
    for _, p in ipairs(PARAMS) do
        if p.key ~= skipKey then
            self:Dirty(p.key)
        end
        self:Dirty(p.key .. "Text")
    end
    self:RefreshStatus()
end

---@private
function CamDebug:RefreshStatus()
    local fit = Config.CameraForAspect(CanvasAspect())
    -- 直接报"本端实际生效值"而不是 Config 上的字面值：t 落在中间时两者不同，
    -- 只看字面值会让人按 Config 去推算画面，越算越乱。
    self.statusLine = string.format(
        "生效 距离%.2f 俯角%.1f 前视%.2f 垂直 FOV%.1f", fit.distance, fit.pitch, fit.lookAhead,
fit.vfov)
    -- FPS 自己拿：面板展开时 GameHud 的性能 Overlay 被让位（两者都想占屏幕上部，同屏只会
    -- 互相盖掉），调构图时不能连帧数都没得看。
    local fps = -1
    local tm = GetSubsystem("Time")
    if tm and tm.GetFramesPerSecond then
        fps = tm:GetFramesPerSecond()
    end
    self.statusLine2 = string.format("aspect%.2f t%.2f | FPS %.0f",
        fit.aspect or 0.0, fit.t or 0.0, fps)
    self:Dirty("statusLine")
    self:Dirty("statusLine2")
end

--- 面板是否占着屏（main.lua 据此吞掉走位输入）。
```

```
---@return boolean
function CamDebug:IsModal()
    return self.showPanel
end

---@param b boolean
function CamDebug:SetOpen(b)
    self.showPanel = b and true or false
    self.showTrigger = not self.showPanel
    self:Dirty("showPanel")
    self:Dirty("showTrigger")
    if self.showPanel then
        -- 展开瞬间整体刷一遍：面板收起期间数据模型没被拉取过，确保看到的都是当前值
        self:Apply(nil)
    end
end

--- 回到面板创建那一刻的值。
function CamDebug:Reset()
    for _, p in ipairs(PARAMS) do
        p.set(self.defaults[p.key])
    end
    self:Apply(nil)
end

--- 把当前值以可直接粘回 Config.lua 的形式打到日志——这是手机上调参的出口。
function CamDebug:Dump()
    local lines = { "-- FireGame CamDebug dump" }
    for _, p in ipairs(PARAMS) do
        lines[#lines + 1] = string.format("%s = %s", p.src, string.format(p.fmt, p.get()))
    end
    local fit = Config.CameraForAspect(CanvasAspect())
    lines[#lines + 1] = string.format("-- 生效: distance=%.2f pitch=%.1f lookAhead=%.2f vfov=%.1f
(aspect=%.2f t=%.2f)",
        fit.distance, fit.pitch, fit.lookAhead, fit.vfov, fit.aspect or 0.0, fit.t or 0.0)
    print(table.concat(lines, "\n"))
end

return CamDebug
-- Scripts/game/Dog.lua
--
-- 主角的宠物狗：一个友方伴随单位。刻意独立于 Enemy 的 self.units（不注册进敌人表），
-- 于是天然获得三条隔离——玩家自动吸附与子弹扫掠只遍历 enemy.units 且只认 pirate/boat/
-- bigboat，够不着狗；波次清怪 ThreatCount 不含狗；爆桶连锁 damage 也只遍历 enemy.units
-- 不误伤狗。狗与外界只有两处接触：
--   * 主动咬海盗：直接对海盗 unit 调 TakeDamage（静默击杀——海盗照样死、照样播死亡动画，
--     但不进 onScore，不加分数/combo/经验、不触发 hit-stop）；
--   * 被大船炮弹炸飞：Enemy:CannonImpact 命中范围内时回调 Dog:HitByExplosion，狗被掀上天
--     翻滚几圈再落地起身。狗无血量，炸不死（纯搞笑反馈）。
-- 海盗不会主动攻击狗（突袭型 AI 只追 player.spot，逻辑未动）。
--
```

```
-- 抖音包无物理/导航，移动是 Lua 直线推进 + 矩形钳制；炸飞靠 Lua 抛物线积分 + 多轴翻滚。
-- 面向约定与海盗一致：角色模型原生朝 -z，朝目标 (dx,dz) 的 yaw = atan(dx,-dz)，取负角度绕 UP。

local Config = require("game.Config")
local Materials = require("game.Materials")
local DogLaunch = require("game.DogLaunch")

---@class Dog 宠物狗（行为 + 生命周期）
local Dog = {}
Dog.__index = Dog

local function distXZ(ax, az, bx, bz)
    local dx, dz = ax - bx, az - bz
    return math.sqrt(dx * dx + dz * dz)
end

---@param deps { scene: Scene, player: Player, enemies: Enemy, audio: Audio }
---@return Dog
function Dog.Build(deps)
    local self = setmetatable({}, Dog)
    self.scene = deps.scene
    self.player = deps.player
    self.enemies = deps.enemies
    self.audio = deps.audio
    self.cfg = Config.DOG

    local root = self.scene:CreateChild("Pets")
    local node = root:CreateChild("Dog")
    self.node = node

    local model = node:CreateComponent("AnimatedModel")
    model:SetModel(GetResource("Model", Config.DogModel()))
    model:SetMaterial(Materials.Character())   -- 与主角共用 Zombie_Atlas 图集材质
    model:SetCastShadows(false)  -- 与主角一致：不投影（投影烘在 prefab 里归 true，运行期显式关掉）
    model:SetUpdateInvisible(false)

    self.anim = node:CreateComponent("AnimationController")

    -- 地面 XZ（self.x/self.z）。狗 node 原点在四脚底面，贴沙面 SAND_Y；追击时海盗多已在
    -- 滩头，咬涉水海盗靠 XZ 判距，狗不必真下海。离地高度与“被炸飞/落地起身”整段生命周期
    -- 已拆进 game/DogLaunch.lua（self.launchCtl），Dog 不再持有 self.h。
    self.x, self.z = 0.0, 0.0
    self.yaw = 0.0
    self.dt = 0.0           -- 本帧 dt，供 FaceDir 限速转向用（Update 每帧刷新）
    self.state = "wander"
    self.biteT = 0.0        -- 咬击冷却
    self.barkT = 0.0        -- 吠叫冷却（仅追击/攻击海盗时刷，防过频）
    self.wanderT = 0.0      -- 当前游荡目标点的停留/重选倒计时
    self.wx, self.wz = 0.0, 0.0
    self.lockedTarget = nil  -- 目标粘滞：当前锁定要咬的海盗 unit，防止等距横跳
```

```
    self.curAnim = nil      -- 当前循环动画名，避免每帧重复 Play
    self.restT = 0.0        -- 就地休息计时（>0 时不移动不索敌）
    self.restReturn = false -- 波次清完：是否正在“跑回主角身边准备休息”
    self.returning = false  -- 跟随溜达：是否处于“正在跑回主角身边”（followRadius/followStopRadius 迟滞）
    self.launchCtl = DogLaunch.New(self)   -- 炸飞/起身的独立状态机（见 game/DogLaunch.lua）

    self:PlaceAtSpawn()
    self:EnsureAnim("Idle")
    return self
end

function Dog:PlaceAtSpawn()
    local p = (self.player and self.player.spot) or Config.PLAYER_SPAWN
    self.x = p.x + self.cfg.spawnOffset.x
    self.z = p.z + self.cfg.spawnOffset.z
    self:ClampXZ()
    self.yaw = Config.PIRATE_YAW   -- 默认朝海（与主角出生朝向一致）
    self.node:SetPosition(Vector3(self.x, Config.SAND_Y, self.z))
    self.node:SetRotation(Quaternion(self.yaw, Vector3.UP))
end

--- 把 (x,z) 钳进可活动区：x 用主角可走范围，z 允许略微探到水线去咬刚上岸的海盗，
--- 但不许深入海里、也不绕到塔后村子。下界按“这个 x 处的岸线 - 探水余量”算：
--- 先钳 x 再算岸线（顺序反了会拿到一个走不到的 x，湾里那几米就白让了）。
function Dog:ClampPair(x, z)
    if x < Config.WALK_MIN_X then
        x = Config.WALK_MIN_X
    elseif x > Config.WALK_MAX_X then
        x = Config.WALK_MAX_X
    end
    local zmin = Config.ShoreZ(x) - 1.5
    if z < zmin then
        z = zmin
    elseif z > Config.WALK_MAX_Z then
        z = Config.WALK_MAX_Z
    end
    return x, z
end

function Dog:ClampXZ()
    self.x, self.z = self:ClampPair(self.x, self.z)
end

function Dog:SetGround()
    self.node:SetPosition(Vector3(self.x, Config.SAND_Y, self.z))
end

--- 朝 (dx,dz) 方向转向（与海盗同一 yaw 约定），沿最短弧限速逼近，不瞬切。
--- 三层防抖：
---   * 死区 faceMinDist：极近距下 atan 输入是亚分米噪声向量，直接不更新；
```

```
---   * faceTurnEps：已对准目标时不再微调，消除对准后的微幅摆抖；
---   * faceTurnRate 限速：即便某帧 desired 在 0/±180 边界跳变（目标与狗近乎正前/正后共线），
---     也只转有限角度，把“瞬时反向”摊成平滑转身，不再逐帧来回乱切。
function Dog:FaceDir(dx, dz)
    if dx == 0.0 and dz == 0.0 then return end
    if dx * dx + dz * dz < self.cfg.faceMinDist * self.cfg.faceMinDist then return end
    local desired = -math.deg(math.atan(dx, -dz))
    -- 最短弧：把 desired 相对当前 yaw 折到 [-180,180)，避免绕远路
    local delta = ((desired - self.yaw + 180) % 360) - 180
    if math.abs(delta) < self.cfg.faceTurnEps then return end
    local maxTurn = self.cfg.faceTurnRate * (self.dt or 0)
    if delta > maxTurn then
        delta = maxTurn
    elseif delta < -maxTurn then
        delta = -maxTurn
    end
    self.yaw = ((self.yaw + delta + 180) % 360) - 180
    self.node:SetRotation(Quaternion(self.yaw, Vector3.UP))
end

function Dog:EnsureAnim(name)
    if self.curAnim == name then return end
    self.curAnim = name
    -- PlayExclusive：同层其余动画淡出。切勿用 Play——循环态（Idle/Walk/Run/Idle_2）不会
    -- 自然结束，Play 不关旧态会让多个循环动画叠在一起满权重混播（休息后走动仍显 Idle_2）。
    self.anim:PlayExclusive(Config.DogAnim(name), 0, true, 0.15)
end

--- 吠一声（仅在追击/攻击海盗时触发；狗无 audio 实例时静默，不影响主流程）。
function Dog:Bark()
    if self.audio then
        self.audio:Play("dogBark")
    end
end

--- 喘气一声（一波清完休息时播；无 audio 实例时静默）。
function Dog:Pant()
    if self.audio then
        self.audio:Play("dogPant")
    end
end

--- 波次清完的入口：狗先跑回主角身边，到位后由 Update 的 restReturn 分支调用 StartRest 就地
--- 休息。空中/起身期间不插队（等落地后主状态机自然接管，这波就不休息了）。
function Dog:GoRest()
    if self.launchCtl:IsActive() then return end
    self.restReturn = true
end

--- 就地休息 restDuration 秒，播一次喘气（跑回主角身边到位后调用）。
```

```
function Dog:StartRest()
    if self.launchCtl:IsActive() then return end
    self.restT = self.cfg.restDuration
    self.state = "rest"
    self:SetGround()
    -- 强制起播 Idle：不走 EnsureAnim 的 curAnim 守卫。上一段可能刚播完一次性 Attack（bite）
    -- 或从 recover 收尾出来，curAnim 记账与实际播放状态不同步，走守卫会让它不真正起播。
    self.curAnim = "Idle"
    self.anim:PlayExclusive(Config.DogAnim("Idle"), 0, true, 0.1)
    self:Pant()
end

--- 视野内最近的存活海盗。只挑 kind=="pirate"（不咬船、不碰爆桶），返回 unit 与抬到躯干
--- 中段的命中世界点（伤害/特效都打胸口，与 Enemy:NearestTarget 同一套 combat 瞄点约定）。
--- 目标粘滞：若当前 lockedTarget 仍存活且在范围内，除非新目标明显更近（距离×stickFactor
--- 仍比锁定目标距离小），否则保持锁定，防止两个等距海盗引发逐帧 A↔B 横跳。
---@return table|nil, Vector3|nil
function Dog:FindPirate()
    local best, bestD, bestPos = nil, self.cfg.aggroRadius, nil
    local units = self.enemies:GetUnits()
    for _, u in ipairs(units) do
        if u.alive and u.node and u.kind == "pirate" then
            local p = u.node:GetWorldPosition()
            local d = distXZ(self.x, self.z, p.x, p.z)
            if d <= bestD then
                best, bestD = u, d
                bestPos = Vector3(p.x, p.y + Config.PIRATE_HIT_HEIGHT, p.z)
            end
        end
    end

    -- 粘滞判定：锁定目标仍有效时，仅当新目标明显更近才切换
    local locked = self.lockedTarget
    if locked and locked.alive and locked.node and best and locked ~= best then
        local lp = locked.node:GetWorldPosition()
        local ld = distXZ(self.x, self.z, lp.x, lp.z)
        if ld <= self.cfg.aggroRadius and ld * self.cfg.stickFactor <= bestD then
            -- 锁定目标仍在范围且“折算后”不算太远（未被子击穿）→ 保持锁定
            best = locked
            bestD = ld
            bestPos = Vector3(lp.x, lp.y + Config.PIRATE_HIT_HEIGHT, lp.z)
        end
    end

    self.lockedTarget = best
    return best, bestPos
end

-----------------------------------------------------------------------
-- 帧更新
```

```
-----------------------------------------------------------------------

---@param dt number
function Dog:Update(dt)
    -- 在空中/落地起身期间由 launch 状态机整体接管（推进位移/翻滚/回正），主状态机让路。
    if self.launchCtl:Update(dt) then
        return
    end

    local c = self.cfg
    self.dt = dt   -- 供 FaceDir 按 faceTurnRate*dt 限速转向用
    -- 波次刚清完：先跑回主角身边，到位再就地休息。途中若冒出新海盗，放弃“回家休息”转去撕咬。
    if self.restReturn then
        local p = (self.player and self.player.spot) or Config.PLAYER_SPAWN
        local d = distXZ(self.x, self.z, p.x, p.z)
        if d <= c.restArriveRadius then
            self.restReturn = false
            self:StartRest()
            return
        end
        if self:FindPirate() then
            self.restReturn = false          -- 有新目标：落到下面正常索敌/撕咬
        else
            local step = c.speed * dt
            local inv = step / d
            self.x = self.x + (p.x - self.x) * inv
            self.z = self.z + (p.z - self.z) * inv
            self:ClampXZ()
            self:FaceDir(p.x - self.x, p.z - self.z)
            self:SetGround()
            self:EnsureAnim("Run")
            return
        end
    end

    -- 已到位休息：钉在原地、待机、不索敌不移动，计时到了自动恢复溜达/追击。
    if self.restT > 0.0 then
        self.restT = self.restT - dt
        self.state = "rest"
        self:SetGround()
        self:EnsureAnim("Idle")
        return
    end

    self.biteT = self.biteT - dt
    self.barkT = self.barkT - dt
    local target, tpos = self:FindPirate()

    if target then
        local d = distXZ(self.x, self.z, tpos.x, tpos.z)
```

```
        if d <= c.biteRange then
            self.state = "bite"
            self:FaceDir(tpos.x - self.x, tpos.z - self.z)
            self:SetGround()
            if self.biteT <= 0.0 then
                self.biteT = c.biteInterval
                self.anim:PlayExclusive(Config.DogAnim(c.biteAnim), 0, false, 0.05)
                self.curAnim = nil                          -- 让下段移动重新起循环动画
                target:TakeDamage(c.biteDamage, tpos, false)   -- 静默击杀：不走 onScore
            end
        else
            self.state = "chase"
            local step = c.speed * dt
            local inv = step / d
            self.x = self.x + (tpos.x - self.x) * inv
            self.z = self.z + (tpos.z - self.z) * inv
            self:ClampXZ()
            self:FaceDir(tpos.x - self.x, tpos.z - self.z)
            self:SetGround()
            self:EnsureAnim("Run")
            if self.barkT <= 0.0 then          -- 冲去咬的路上反复吠
                self.barkT = c.barkCooldown
                self:Bark()
            end
        end
    else
        self:UpdateWander(dt)
    end
end

--- 无海盗时的溜达：离主太远先回家；否则在主角周围随机挑点慢逛，到位就原地待机。
function Dog:UpdateWander(dt)
    local c = self.cfg
    self.state = "wander"
    local p = (self.player and self.player.spot) or Config.PLAYER_SPAWN
    local dHome = distXZ(self.x, self.z, p.x, p.z)

    -- 跟随/溜达两模式的迟滞：超过 followRadius 才启动“回家”，一旦启动就连续跑、直到距主角
    -- 小于 followStopRadius 才放开。否则在 followRadius 边界会逐帧在“朝主人跑(朝内)”与“朝
    -- 漫游点走(朝外)”间切换，两个朝向几乎相反——这正是快速来回乱切的一个成因。
    if self.returning then
        if dHome <= c.followStopRadius then self.returning = false end
    elseif dHome > c.followRadius then
        self.returning = true
    end

    if self.returning then
        local step = c.speed * dt
        local inv = step / dHome
        self.x = self.x + (p.x - self.x) * inv
```

```
        self.z = self.z + (p.z - self.z) * inv
        self:ClampXZ()
        self:FaceDir(p.x - self.x, p.z - self.z)
        self:SetGround()
        self:EnsureAnim("Run")
        return
    end

    -- 只在计时到期时重选漫游目标点：到达后先原地待机等到点再挑下一个。旧代码把“与目标点距离<0.5”
    -- 也当成重选触发条件，而新点是绕主角按 0..wanderRadius 任意半径抽的——一旦抽到离狗很近的点
    -- （<0.5），下一帧又判“到达”再抽，FaceDir 每帧被甩向全新的随机方向，表现为快速来回乱切。
    self.wanderT = self.wanderT - dt
    if self.wanderT <= 0.0 then
        local picked = false
        for _ = 1, 8 do
            local a = math.random() * math.pi * 2.0
            local r = c.wanderMinDist + math.random() * (c.wanderRadius - c.wanderMinDist)
            local nx, nz = self:ClampPair(p.x + math.cos(a) * r, p.z + math.sin(a) * r)
            -- 新点需离狗当前位置足够远，否则走两步即“到达”、几乎立刻翻面
            if distXZ(self.x, self.z, nx, nz) >= c.wanderMinDist then
                self.wx, self.wz = nx, nz
                picked = true
                break
            end
        end
        if not picked then
            -- 主角贴墙/角落、8 次都抽不到足够远的点时的兜底：给一个固定的侧向目标，避免空转
            self.wx, self.wz = self:ClampPair(self.x - c.wanderMinDist, self.z)
        end
        self.wanderT = 2.0 + math.random() * 3.0
    end

    local d = distXZ(self.x, self.z, self.wx, self.wz)
    if d > c.wanderArrive then
        local step = c.wanderSpeed * dt
        local inv = step / d
        self.x = self.x + (self.wx - self.x) * inv
        self.z = self.z + (self.wz - self.z) * inv
        self:ClampXZ()
        self:FaceDir(self.wx - self.x, self.wz - self.z)
        self:SetGround()
        self:EnsureAnim("Walk")
    else
        self:SetGround()
        self:EnsureAnim("Idle")
    end
end

--------------------------------------------------------------------------
-- 被炮弹炸飞（程序化抛物线 + 多轴翻滚 + 落地起身）：整段生命周期见 game/DogLaunch.lua。
```

```
-- Enemy:CannonImpact 仍打到 Dog:HitByExplosion，这里只转发给内部的 launch 状态机；
-- blastPos 为炮弹落点（世界，y 传地面即可），radius 为杀伤半径。
--------------------------------------------------------------------------
---@param blastPos Vector3
---@param radius number
function Dog:HitByExplosion(blastPos, radius)
    self.launchCtl:HitByExplosion(blastPos, radius)
end

--------------------------------------------------------------------------
-- 换局
--------------------------------------------------------------------------

function Dog:Reset()
    self.launchCtl:Reset()
    self.restT = 0.0
    self.restReturn = false
    self.returning = false
    self.biteT = 0.0
    self.barkT = 0.0
    self.wanderT = 0.0
    self.lockedTarget = nil
    self.curAnim = nil
    self.state = "wander"
    self:PlaceAtSpawn()
    self:EnsureAnim("Idle")
end

return Dog
-- Scripts/game/DogLaunch.lua
--
-- 宠物狗"被大船炮弹炸飞"的独立状态机：程序化抛物线 + 多轴翻滚 + 落地起身回正。
-- 从 game/Dog.lua 里拆出来，是为了把这段与索敌/移动完全正交的生命周期单点收拢——
-- Dog 只在 Enemy:CannonImpact 命中时转发一句 HitByExplosion，其余（空中推进、翻滚、
-- 起身）都归本模块，空中期间整体接管狗节点，落地后交回 Dog 的主状态机。
--
-- 它不改血量、不判存活：狗炸不死（纯搞笑反馈）。
-- 抖音包无物理，绝不做刚体——竖直靠 vy/重力积分，水平靠带阻尼的弹出位移，翻滚靠每帧
-- 在当前世界朝向上前乘一个绕斜轴的小角增量。坐标/朝向约定与 Dog 一致：node 原点在四脚
-- 底面、贴沙面 Config.SAND_Y。
--
-- 依赖注入：构造时传入 dog，读写 dog 的 node/x/z/yaw/state/curAnim/biteT/cfg.launch，
-- 并调用 dog:ClampXZ / dog:SetGround / dog:EnsureAnim / dog:Bark 与 dog.anim。本模块不
-- require Dog（避免循环），只 require Config 取 SAND_Y / DogAnim。

local Config = require("game.Config")

---@class DogLaunch 炸飞/起身生命周期状态机
local DogLaunch = {}
DogLaunch.__index = DogLaunch
```

```
local function distXZ(ax, az, bx, bz)
    local dx, dz = ax - bx, az - bz
    return math.sqrt(dx * dx + dz * dz)
end

---@param dog Dog 宿主（提供 node/x/z/yaw/cfg.launch 与 ClampXZ/SetGround/EnsureAnim/Bark/anim）
---@return DogLaunch
function DogLaunch.New(dog)
    return setmetatable({
        dog = dog,
        launch = nil,       -- 非 nil 表示在空中：{ vy, dirx, dirz, axis }
        recoverT = 0.0,     -- 落地起身计时
        h = 0.0,            -- 离地高度（正常 0，被炸飞时走抛物线）
    }, DogLaunch)
end

--- 是否正处于"在空中 / 落地起身"期间——Dog 的主状态机据此让路，休息/索敌也不该插队。
---@return boolean
function DogLaunch:IsActive()
    return self.launch ~= nil or self.recoverT > 0.0
end

--- 回到无炸飞状态（换局/重生用）。
function DogLaunch:Reset()
    self.launch = nil
    self.recoverT = 0.0
    self.h = 0.0
end

--- 由 Enemy:CannonImpact 经 Dog:HitByExplosion 转发进来：blastPos 为炮弹落点（世界，
--- y 传地面即可），radius 为杀伤半径。只有水平距离在半径内、且此刻不在空中/不在起身时才掀飞。
---@param blastPos Vector3
---@param radius number
function DogLaunch:HitByExplosion(blastPos, radius)
    if self:IsActive() then return end
    local dog = self.dog
    if distXZ(dog.x, dog.z, blastPos.x, blastPos.z) <= radius then
        self:Launch(blastPos)
    end
end

--- 起跳：算出弹出方向与初速，切到 launched 态，播 Run_Jump 并惨叫一声。
---@param blastPos Vector3
function DogLaunch:Launch(blastPos)
    local dog = self.dog
    local lc = dog.cfg.launch
    -- 弹出方向 = 由爆炸点指向狗（被气浪推开），取不到方向就随机一个
    local dx, dz = dog.x - blastPos.x, dog.z - blastPos.z
    local d = math.sqrt(dx * dx + dz * dz)
```

```
    if d < 1e-4 then
        local a = math.random() * math.pi * 2.0
        dx, dz, d = math.cos(a), math.sin(a), 1.0
    end
    self.launch = {
        vy = lc.upSpeed,
        dirx = dx / d * lc.forwardKick,
        dirz = dz / d * lc.forwardKick,
        -- 斜翻轴：不是纯水平/纯竖直，滚起来像被甩飞而非原地打转
        axis = Vector3(0.8, 0.35, 0.6):Normalized(),
    }
    self.h = 0.0
    dog.state = "launched"
    dog.biteT = 0.0
    dog.curAnim = nil
    dog.anim:PlayExclusive(Config.DogAnim("Run_Jump"), 0, true, 0.05)
    dog:Bark()   -- 被炸飞时惨叫一声
end

--- 每帧推进：在空中则积分抛物线 + 翻滚，落地转入起身；起身则平滑滚回四脚着地。
--- 返回 true 表示本帧被本模块接管（Dog 应跳过主状态机）。
---@param dt number
---@return boolean
function DogLaunch:Update(dt)
    if self.launch then
        self:UpdateLaunch(dt)
        return true
    end
    if self.recoverT > 0.0 then
        self:UpdateRecover(dt)
        return true
    end
    return false
end

function DogLaunch:UpdateLaunch(dt)
    local dog = self.dog
    local lc = dog.cfg.launch
    local l = self.launch
    -- 竖直：上升→重力回落，回到 h<=0 即落地
    l.vy = l.vy - lc.gravity * dt
    self.h = self.h + l.vy * dt
    -- 水平：沿弹出方向位移，带阻尼衰减（气浪推一下就停）
    dog.x = dog.x + l.dirx * dt
    dog.z = dog.z + l.dirz * dt
    local damp = 1.0 - 2.0 * dt
    if damp < 0.0 then damp = 0.0 end
    l.dirx, l.dirz = l.dirx * damp, l.dirz * damp
    dog:ClampXZ()
```

```
    if self.h <= 0.0 then
        self.h = 0.0
        self.launch = nil
        self.recoverT = lc.recoverTime
        dog:SetGround()
        return
    end

    dog.node:SetPosition(Vector3(dog.x, Config.SAND_Y + self.h, dog.z))
    -- 绕世界斜轴叠加一个小角增量 → 连续翻滚（不累积误差，每帧在当前朝向上前乘）
    local delta = Quaternion(lc.spinSpeed * dt, l.axis)
    dog.node:SetRotation(delta * dog.node:GetWorldRotation())
end

--- 落地起身：把翻滚留下的朝向平滑滚回四脚着地（回正到 dog.yaw），期间待机动画。
function DogLaunch:UpdateRecover(dt)
    local dog = self.dog
    self.recoverT = self.recoverT - dt
    local up = Quaternion(dog.yaw, Vector3.UP)
    dog.node:SetRotation(dog.node:GetWorldRotation():Slerp(up, math.min(1.0, dt * 6.0)))
    dog:SetGround()
    dog:EnsureAnim("Idle")
    if self.recoverT <= 0.0 then
        dog.node:SetRotation(Quaternion(dog.yaw, Vector3.UP))
        dog.state = "wander"
    end
end

return DogLaunch
-- Scripts/game/Voice.lua
--
-- 自定义语音（麦克风录一句 → 变声回放）：玩法情绪钩子的执行器。
--   槽位：lose（防线失守播你的吐槽）、killstreak（连杀里程碑播你的战吼）。
--
-- 链路分工（重要）：
--   * 引擎侧（MinigameSDK）只发请求收事件——RecordVoice/PlayUserAudio/StopUserAudio，
--     每个请求返回 id，结果经 "MinigameSdkResult" 事件回来（data.Kind 对照
--     MINIGAME_REQUEST_KIND 表）。
--   * 宿主 JS 侧（minigame_sdk.js）拿 recorderManager 录 aac 存进用户目录
--     （每次全新路径，槽位→路径的映射持久化在宿主 storage，跨局/跨会话存活），
--     回放优先走 WebAudio 解码并按峰值归一放大（手机麦电平太低，InnerAudioContext
--     的 volume 上限 1.0 拉不动），按档位变声；宿主不支持时回落 InnerAudioContext。
--     引擎的 Sound 解码器全程不参与（它只认 ogg/wav）。
--   * 编辑器/桌面宿主没有 SDK（IsAvailable()=false，请求立即失败）——UI 入口
--     由 Hud 按 voiceAvailable 隐藏，本模块的调用全部静默返回，不会报错。
--
-- 音色：playbackRate 只能取宿主档位（0.5/0.8/1.0/1.25/1.5/2.0），配置在
-- Config.VOICE_FILTERS；循环切换（CycleFilter）给试听/回放共用。

local Config = require("game.Config")
```

```
local Tracker = require("game.Tracker")

---@class Voice 自定义语音的请求编排与状态
local Voice = {}
Voice.__index = Voice

---@param scene Scene
---@param deps { hud: Hud|nil }|nil
---@return Voice
function Voice.Build(scene, deps)
    local self = setmetatable({}, Voice)
    self.hud = deps and deps.hud
    self.sdk = GetSubsystem("MinigameSDK")

    self.filterIdx = math.min(math.max(Config.VOICE_DEFAULT_FILTER, 1), #Config.VOICE_FILTERS)
    self.recording = false       -- 是否有录音请求在途（防止重入）
    self.recordDeadline = 0      -- 录音看门狗的剩余秒数（0 = 没有在途请求）
    self.previewing = false      -- 是否有试听请求在途（结果回写 HUD 状态行）
    self.lastSlot = nil          -- 最近录成功的槽位（试听默认播它）
    self.pendingSlot = nil       -- 本次录音请求的槽位（结果回来时才落进 lastSlot）
    self.pendingClearSlot = nil  -- 本次清除请求的槽位（成功后才清 lastSlot）
    -- 玩法触发回放（Play）的回落约定：宿主回放失败 → 播预置旁白。只认“这一次请求”的
    -- 失败：JS 侧新的回放会顶掉旧的并以 success=false("playback stopped") 结算，若不看 id，
    -- 被顶掉的那次会误触发回落，出现“既有你的声音又有 Kenney 旁白”的双响。
    self.pendingPlayId = 0       -- 0 = 没有待回落的玩法回放
    self.pendingFallback = nil   -- 该次回放失败时要补播的预置旁白（fun）

    -- 录音/试听结果都从这里回来（BEGINFRAME 边界派发，见 MinigameSDK.cpp）
    SubscribeToEvent("MinigameSdkResult", function(data)
        self:OnSdkResult(data)
    end)

    -- 录音看门狗：宿主若既不给结果也不报错（shim 侧已见到过一次这种吞回调），面板会永远
    -- 停在“录音中”。shim 自带 duration+30s 的结算，这里再留几秒余量，谁先到谁解锁。
    SubscribeToEvent("Update", function(data)
        self:OnUpdate(data.TimeStep)
    end)

    -- UI 侧初始状态：入口按宿主能力显隐（编辑器/桌面整块不渲染），音色名先显示默认档
    if self.hud then
        if self.hud.SetVoiceAvailable then
            self.hud:SetVoiceAvailable(self:IsAvailable())
        end
        if self.hud.SetVoiceFilterName then
            self.hud:SetVoiceFilterName(Config.VOICE_FILTERS[self.filterIdx].name)
        end
    end
    return self
end
```

```
---@return boolean 宿主 SDK 是否可用（编辑器里为 false，UI 据此隐藏入口）
function Voice:IsAvailable()
    return self.sdk ~= nil and self.sdk:IsAvailable()
end

--- 当前音色的 playbackRate（宿主档位值）。
---@return number
function Voice:CurrentRate()
    return Config.VOICE_FILTERS[self.filterIdx].rate
end

--- 循环切换音色（UI 按钮），返回新音色名。
---@return string
function Voice:CycleFilter()
    self.filterIdx = self.filterIdx % #Config.VOICE_FILTERS + 1
    return Config.VOICE_FILTERS[self.filterIdx].name
end

--- 录一句（面板按钮）。到时自动停（上限 Config.VOICE_MAX_SEC），结果走事件。
---@param slot string Config.VOICE_SLOT_*
function Voice:Record(slot)
    if not self:IsAvailable() then
        return
    end
    if self.recording then
        return  -- 录音中再点：静默忽略（shim 侧也有闸）
    end
    self.recording = true
    self.pendingSlot = slot
    -- 上限秒数 + 余量：shim 侧在 duration+30s 结算，这个只兜“连 shim 都没回”的情况
    self.recordDeadline = Config.VOICE_MAX_SEC + 35
    if self.hud and self.hud.SetVoiceRecording then
        self.hud:SetVoiceRecording(true)
    end
    self.sdk:RecordVoice(slot, Config.VOICE_MAX_SEC)
end

--- 录音看门狗（订阅见 Build）：超时还在挂着就解锁面板，别让玩家对着“录音中”干等。
---@param dt number
function Voice:OnUpdate(dt)
    if not self.recording or self.recordDeadline <= 0 then
        return
    end
    self.recordDeadline = self.recordDeadline - (dt or 0)
    if self.recordDeadline > 0 then
        return
    end
    self.recordDeadline = 0
    self.recording = false
    print("Voice: recording never settled; unlocking the panel")
```

```
    if self.hud then
        if self.hud.SetVoiceRecording then
            self.hud:SetVoiceRecording(false)
        end
        if self.hud.SetVoiceStatus then
            self.hud:SetVoiceStatus("录音没响应，再试一次")
        end
    end
end

--- 试听（面板按钮，按槽位）：指定槽位就播它（没录过则报“还没录过音”）；
--- 不指定时回落最近录的槽位（录完自动回放走这条路，lastSlot 刚被置为本次录音的槽）。
---@param slot string|nil Config.VOICE_SLOT_*
function Voice:Preview(slot)
    if not self:IsAvailable() then
        return
    end
    slot = slot or self.lastSlot or Config.VOICE_SLOT_LOSE
    self.previewing = true
    -- 新的回放顶掉上一条：那条玩法回放是被主动打断的，不该再回落预置旁白
    self.pendingPlayId = 0
    self.pendingFallback = nil
    self.sdk:PlayUserAudio(slot, self:CurrentRate())
end

--- 玩法触发回放（失守吐槽/连杀战吼）：录过就播你的，没录过回落预置旁白。
--- 触发点不弹 UI（战场上突然冒“没录过音”比没声音更糟），所以回落靠宿主回放失败事件。
---@param slot string Config.VOICE_SLOT_*
---@param fallback fun()|nil 宿主里没有这段录音时补播的那句旁白
function Voice:Play(slot, fallback)
    if not self:IsAvailable() then
        if fallback then
            fallback()
        end
        return
    end
    self.pendingFallback = fallback
    self.pendingPlayId = self.sdk:PlayUserAudio(slot, self:CurrentRate())
end

--- 停掉当前回放（换局时调，避免上一局的语音拖进新局）。
function Voice:Stop()
    self.pendingPlayId = 0
    self.pendingFallback = nil
    self.previewing = false
    if not self:IsAvailable() then
        return
    end
    self.sdk:StopUserAudio()
end
```

```
--- 清除某个槽位的录音（面板按钮）：删掉宿主里那句，该槽位从此回到预置旁白。
--- 录过再清、清完再录都行；没录过就点清除也算成功（本来就是没录过的状态）。
---@param slot string Config.VOICE_SLOT_*
function Voice:Clear(slot)
    if not self:IsAvailable() then
        return
    end
    if self.recording then
        return  -- 录音中不清（面板是模态的，正常走不到）
    end
    -- 先停下回放：不去删一个正被宿主音频上下文打开的文件
    self:Stop()
    self.pendingClearSlot = slot
    self.sdk:ClearUserAudio(slot)
end

--- MinigameSdkResult 事件分发：录音/试听结果回写 HUD 状态行。
function Voice:OnSdkResult(data)
    local kind = data.Kind
    if kind == MINIGAME_REQUEST_KIND.RECORD_VOICE then
        self.recording = false
        self.recordDeadline = 0
        if self.hud and self.hud.SetVoiceRecording then
            self.hud:SetVoiceRecording(false)
        end
        if data.Success then
            -- Payload 形如 {"durationMs":2143}；槽位由发起方记在 pendingSlot
            self.lastSlot = self.pendingSlot or self.lastSlot
            if self.hud and self.hud.SetVoiceStatus then
                self.hud:SetVoiceStatus("录好了！听听变声效果")
            end
            Tracker.Report("voice_record", { slot = self.pendingSlot, ok = true })
            -- 录完立刻回放一遍：变声的听感就是这功能的全部，别让玩家自己再点一次
            self:Preview()
        else
            -- 宿主回来的失败原文（errMsg/errCode）进日志：真机上只有这一条线索能把
            -- “没授权”“设备忙”“格式不支持”区分开，状态行只放玩家能动作的那一句。
            local reason = tostring(data.Payload or "")
            print("Voice: recording failed: " .. reason)
            Tracker.Report("voice_record", {
                slot = self.pendingSlot,
                ok = false,
                denied = string.find(reason, "permission denied", 1, true) ~= nil,
            })
            if self.hud and self.hud.SetVoiceStatus then
                if string.find(reason, "permission denied", 1, true) then
                    self.hud:SetVoiceStatus("麦克风没开权限：小游戏和抖音 App 都要允许")
                else
                    -- 日志里已有宿主原话（上面那条 print），状态行给玩家能动作的文案。
```

```
                    self.hud:SetVoiceStatus("录音失败，再试一次")
                end
            end
        end
    elseif kind == MINIGAME_REQUEST_KIND.CLEAR_USER_AUDIO then
        local slot = self.pendingClearSlot
        self.pendingClearSlot = nil
        Tracker.Report("voice_clear", { slot = slot, ok = data.Success })
        if data.Success then
            -- 试听的默认槽位不能还指向一个已经没了的文件
            if self.lastSlot == slot then
                self.lastSlot = nil
            end
            if self.hud and self.hud.SetVoiceStatus then
                self.hud:SetVoiceStatus("已清除，改回预置旁白")
            end
        elseif self.hud and self.hud.SetVoiceStatus then
            self.hud:SetVoiceStatus("清除失败，再试一次")
        end
    elseif kind == MINIGAME_REQUEST_KIND.PLAY_USER_AUDIO then
        -- 玩法回放失败（该槽位没录过 / 录音文件被清理）：补播预置旁白。
        -- 按请求 id 认领，被后一次回放顶掉的旧请求在这里对不上，不会误触发。
        if self.pendingPlayId ~= 0 and data.RequestId == self.pendingPlayId then
            local fallback = self.pendingFallback
            self.pendingPlayId = 0
            self.pendingFallback = nil
            if not data.Success and fallback then
                fallback()
            end
        end
        if self.previewing then
            self.previewing = false
            if self.hud and self.hud.SetVoiceStatus then
                if data.Success then
                    self.hud:SetVoiceStatus("就是这效果！")
                else
                    self.hud:SetVoiceStatus("还没录过音，先录一句吧")
                end
            end
        end
    end
end

return Voice
-- Scripts/game/Sidebar.lua
--
-- 抖音侧边栏复访（平台必接合规，2023-11-24 起不接直接驳回）：识别进入场景 +
-- 老用户回归奖励 + 新用户引导，三件事都在这里。
--
-- 链路分工（与 Voice 同一模式）：
```

```
--   * 引擎侧（MinigameSDK）只发请求收事件——GetLaunchInfo / NavigateToSidebar，
--     结果经 "MinigameSdkResult" 事件回来（data.Kind 对照 MINIGAME_REQUEST_KIND 表）。
--   * 宿主 JS 侧（minigame_sdk.js）管 scene 判定（抖音侧边栏 scene=021236）与
--     guideDone 持久化（宿主 storage），payload 形如 {"sidebar":true,"guideDone":false}，
--     JSON 布尔用 string.find 认领（同 Voice 的做法，不引 JSON 库）。
--
-- 行为：
--   * 冷启动查一次场景。从侧边栏进入 → 本会话增益（攻击 xSIDEBAR_DMG_MULT、
--     生命上限 xSIDEBAR_HP_MULT，Waves/Shooting 各自乘）+ 顶部旁白播报；
--     提示每会话只发一次（防 onShow 重查反复刷旁白），数值整会话有效。
--   * 非侧边栏首次进入 → 引导弹窗。弹窗等开始画面收起再出（两层全屏模态叠着
--     只会更黑更吵）；玩家点过「去侧边栏」或「知道了」后宿主 storage 记 guideDone，
--     以后再也不弹（本会话也即时关门，防 storage 写失败时循环打扰）。
--   * 宿主 onShow（引擎 "MinigameShow" 事件）后重查一次：玩家从侧边栏面板点回
--     游戏（进程保活、不冷启动）也走 onShow options.scene，这条路也能领到奖励。
--   * 编辑器/桌面宿主没有 SDK → 整个模块静默（增益恒 1.0、引导不弹）；
--     要在编辑器里验证两条路径用 Config.DEBUG_SIDEBAR。
--
local Config = require("game.Config")
local Tracker = require("game.Tracker")

---@class Sidebar 侧边栏复访：进入场景识别 + 回归奖励 + 引导
local Sidebar = {}
Sidebar.__index = Sidebar

-- 增益查询口：Waves（生命上限）与 Shooting（每发伤害）各调一次。
-- 模块单例未建（编辑器异常加载顺序）或增益未生效时恒 1.0，乘了等于没乘。
function Sidebar.DmgMult()
    return (Sidebar.instance and Sidebar.instance.buff) and Config.SIDEBAR_DMG_MULT or 1.0
end

function Sidebar.HpMult()
    return (Sidebar.instance and Sidebar.instance.buff) and Config.SIDEBAR_HP_MULT or 1.0
end

---@param scene Scene
---@param deps { hud: Hud|nil, getWaves: fun():Waves|nil }|nil
---@return Sidebar
function Sidebar.Build(scene, deps)
    local self = setmetatable({}, Sidebar)
    Sidebar.instance = self
    self.hud = deps and deps.hud
    self.getWaves = deps and deps.getWaves
    self.sdk = GetSubsystem("MinigameSDK")

    self.buff = false           -- 侧边栏回归增益是否生效（会话级，重入播报也由它挡）
    self.narratePending = false -- 增益到账时开始画面还盖着 → 改到收起后再播
    self.pendingGuide = false   -- 查到要引导但开始画面还盖着 → 等收起再弹
    self.guideMarked = false    -- 本会话已引导过（宿主 storage 写失败时的兜底闸）
    self.requestInFlight = false
```

```
    SubscribeToEvent("MinigameSdkResult", function(data)
        self:OnSdkResult(data)
    end)
    -- 宿主回前台（含从侧边栏面板点回来）：重查场景。真机才有此事件，编辑器不触发。
    SubscribeToEvent("MinigameShow", function(_data)
        self:Query(false)
    end)

    -- 编辑器调试模拟（Config.DEBUG_SIDEBAR）：真机链路走不到这里。
    local dbg = Config.DEBUG_SIDEBAR
    if dbg == "reward" then
        self:GrantBuff()
    elseif dbg == "guide" then
        self.pendingGuide = true
    end

    self:Query(false)
    return self
end

---@return boolean 宿主 SDK 是否可用（编辑器里为 false，整条链路静默）
function Sidebar:IsAvailable()
    return self.sdk ~= nil and self.sdk:IsAvailable()
end

--- 发一次场景查询（mark=true 时顺带持久化「引导已做」）。在途时静默忽略：
--- 回包自带全量状态，旧查询的回答与新一轮意图一致，丢一次无害。
---@param mark boolean|nil
function Sidebar:Query(mark)
    if not self:IsAvailable() or self.requestInFlight then
        return
    end
    self.requestInFlight = true
    self.sdk:GetLaunchInfo(mark or false)
end

--- MinigameSdkResult 事件分发（BEGINFRAME 边界）：GET_LAUNCH_INFO 的回包裁决。
function Sidebar:OnSdkResult(data)
    if data.Kind ~= MINIGAME_REQUEST_KIND.GET_LAUNCH_INFO then
        return
    end
    self.requestInFlight = false
    if not data.Success then
        -- 桌面/编辑器（无宿主）也走这里：静默即可，不刷屏。
        return
    end
    local payload = tostring(data.Payload or "")
    local fromSidebar = string.find(payload, '"sidebar":true', 1, true) ~= nil
    local guideDone = string.find(payload, '"guideDone":true', 1, true) ~= nil
```

```
    if fromSidebar then
        self:GrantBuff()
    elseif not guideDone and not self.guideMarked then
        -- 引导等开始画面收起再弹；这里只记意图。
        self.pendingGuide = true
    end
end

--- 开始画面收起（main.lua 的 onStart 回调）后调：把攒下的引导/奖励播报放出来。
function Sidebar:OnGameStart()
    if self.narratePending then
        self.narratePending = false
        self:AnnounceBuff()
    end
    if self.pendingGuide then
        self.pendingGuide = false
        if self.hud and self.hud.OpenSidebarGuide then
            self.hud:OpenSidebarGuide()
        end
    end
end

--- 侧边栏回归增益到账（数值立即生效，播报看开始画面状态）。重入静默：
--- 增益是会话级布尔，第二次调用只可能来自 onShow 重查，数值早已在账上。
function Sidebar:GrantBuff()
    if self.buff then
        return
    end
    self.buff = true
    Tracker.Report("sidebar_revisit", { dmg = Config.SIDEBAR_DMG_MULT, hp =
Config.SIDEBAR_HP_MULT })
    -- 生命增益即时落到当前局：血量唯一真相在 waves.playerHp/playerMaxHp。注意当前局的
    -- playerMaxHp 是增益生效【前】算的（冷启动时 SDK 结果最早下一帧回来，Waves.Reset 早跑完），
    -- 所以这里直接按新上限重算两个字段；后续 Waves:Reset 会经 Sidebar.HpMult() 带上倍率。
    -- 只在满血时抬（冷启动 boot 时必然满血，这是主路径）；局中掉着血回来不白补，下一局
    -- Reset 自然生效——行为可解释，也不给「掉血时切侧边栏白赚回复」留空子。
    local waves = self.getWaves and self.getWaves()
    if waves and waves.playerMaxHp and waves.playerHp == Config.PLAYER_HP then
        waves.playerMaxHp = Config.PLAYER_HP * Config.SIDEBAR_HP_MULT
        waves.playerHp = waves.playerMaxHp
    end
    -- 播报：开始画面盖着会被全屏遮罩吃掉（DOM 在前），改到收起后再播。
    if self.hud and self.hud.IsStartModal and self.hud:IsStartModal() then
        self.narratePending = true
    else
        self:AnnounceBuff()
    end
end

function Sidebar:AnnounceBuff()
    if self.hud and self.hud.Narrate then
```

```
        self.hud:Narrate("侧边栏回归奖励：攻击 +20% 生命 +25%", 4.0)
    end
end

--- 引导弹窗「去侧边栏」（main.lua 的 onSidebarGo 转发）：拉起宿主侧边栏面板 +
--- 持久化已引导。面板拉起后游戏退后台，玩家从侧边栏点回来走 onShow 重查发奖。
function Sidebar:GoSidebar()
    Tracker.Report("sidebar_guide_go", {})
    self:MarkGuideDone()
    if self.hud and self.hud.CloseSidebarGuide then
        self.hud:CloseSidebarGuide()
    end
    if self:IsAvailable() then
        self.sdk:NavigateToSidebar()
    end
end

--- 引导弹窗「知道了」（main.lua 的 onSidebarClose 转发）：只持久化 + 关面板。
function Sidebar:DismissGuide()
    Tracker.Report("sidebar_guide_close", {})
    self:MarkGuideDone()
    if self.hud and self.hud.CloseSidebarGuide then
        self.hud:CloseSidebarGuide()
    end
end

--- 记「引导已做」：宿主 storage 持久化 + 本会话闸（storage 写失败时也不重复弹）。
--- 不走 Query 的在途闸：mark 这一次丢了不是无害（下次冷启动还会再弹引导）。
function Sidebar:MarkGuideDone()
    self.guideMarked = true
    if not self:IsAvailable() then
        return
    end
    self.sdk:GetLaunchInfo(true)
end

return Sidebar
-- Scripts/game/Tracker.lua
--
-- 埋点：把玩法事件送到宿主 analytics 后台（抖音 reportAnalytics / 微信 reportEvent，
-- 由 JS shim 分派，两家都不支持时静默丢弃）。
--
-- 设计约束：
--   * 埋点绝不影响游戏：调用全部 pcall 包裹；宿主 SDK 不可用（编辑器/桌面）时
--     模块整体静默，不打印不报错。
--   * 字段只收扁平一层的 string / number / boolean：宿主事件面板只认扁平值，
--     嵌套表序列化了也没法看，干脆不收。
--   * 事件名即漏斗的一级：boot / game_start / wave / game_over / restart /
--     voice_record / voice_clear；字段是二级维度。
--
```

```
-- 与 JS 侧的分工：boot（冷启动毫秒、平台）由 loading.js 在引擎首帧时直报，
-- 不经本模块——那时 Lua 还没跑起来；其余玩法事件都从这里走。
-- js_error 由 minigame_runtime.js 的 onError/onUnhandledRejection 直报，同理。

local Tracker = {}

local sdk = nil          -- MinigameSDK 子系统（宿主可用时才非空）
local resolved = false   -- 只解析一次：main.lua 加载早于子系统注册完成也没关系，
                         -- 首次 Report 发生在 Build 之后，那时子系统必然就绪

local function Resolve()
    if resolved then
        return
    end
    resolved = true
    sdk = GetSubsystem("MinigameSDK")
end

--- JSON 字符串转义：只处理会破坏 JSON 结构的字符。事件字段全部由游戏侧
--- 控制（不含玩家自由文本），控制字符不做全量转义。
local ESCAPE_PATTERN = '[' .. '"' .. '\\' .. '\n' .. '\r' .. '\t' .. ']'

local function Escape(text)
    return (tostring(text):gsub(ESCAPE_PATTERN, {
        ['"'] = '\\"',
        ['\\'] = '\\\\',
        ['\n'] = '\\n',
        ['\r'] = '\\r',
        ['\t'] = '\\t',
    }))
end

--- 单个字段值 → JSON 字面量。不支持的类型（nil/表/函数）返回 nil，字段被跳过。
local function EncodeValue(value)
    local t = type(value)
    if t == "number" then
        -- inf/nan 不是合法 JSON 数字，落到 0 比丢掉整条事件好
        if value ~= value or value == math.huge or value == -math.huge then
            return "0"
        end
        return tostring(value)
    elseif t == "boolean" then
        return tostring(value)
    elseif t == "string" then
        return '"' .. Escape(value) .. '"'
    end
    return nil
end

--- 扁平字段表 → JSON 对象字符串；空表/全不支持返回 nil（该事件不带参数上报）。
```

```
local function EncodeParams(fields)
    if type(fields) ~= "table" then
        return nil
    end
    local parts = {}
    for k, v in pairs(fields) do
        local encoded = EncodeValue(v)
        if encoded then
            parts[#parts + 1] = '"' .. Escape(k) .. '":' .. encoded
        end
    end
    if #parts == 0 then
        return nil
    end
    return "{" .. table.concat(parts, ",") .. "}"
end

--- 上报一个事件。fields 为扁平表（string/number/boolean），可空。
---@param event string 事件名（漏斗一级）
---@param fields table|nil 事件维度
function Tracker.Report(event, fields)
    Resolve()
    if not sdk or not sdk:IsAvailable() then
        return
    end
    local params = EncodeParams(fields)
    pcall(function()
        if params then
            sdk:ReportAnalytics(event, params)
        else
            sdk:ReportAnalytics(event)
        end
    end)
end

return Tracker
```